

MASGym: A Co-Evolving Red/Blue Security Gym for Multi-Agent LLM Systems with Parameterized Collusion, Orchestrator Compromise, and System-Level Metrics

Thanh Pham*

School of Computing Technologies, RMIT University, Melbourne, Australia

thanhphamchivn@gmail.com

* Corresponding author.

1 Introduction

Progress in the security of large language model (LLM) agents has tracked the quality of the instruments used to measure it. Agent Security Bench (ASB) [78] formalized attacks and defenses across an agent’s operational stages and introduced a utility–security metric $\text{NRP} = \text{PNA} \cdot (1 - \text{ASR})$; AgentDojo [14] evaluates an agent under prompt injection with *deterministic*, state-based `security()` checks, so that a successful injection cannot also corrupt the evaluator; InjecAgent [77] benchmarks indirect prompt injection in tool-integrated agents; and the ToolEmu sandbox [58] lets a language model emulate arbitrary tool execution from specifications, trading fidelity for breadth. These instruments are strong, and they are all single-agent. Their only cross-agent channel is a shared memory store, and none of them exposes a knob for the phenomena that arise when several agents act together.

Deployment has moved in the opposite direction. Production systems such as AutoGen [72], MetaGPT [29], CAMEL [39], ChatDev [55], AgentVerse [9], and Magentic-One [18] are orchestrated *collectives*: a planner directs workers, agents read and write a shared memory, third-party tools are invoked on the critical path, and many tasks are structured so that no single agent can finish them alone. In such systems the security questions that matter are group-level. Do a fraction of agents *collude*? Is the *orchestrator* itself trusted, or has it been compromised? Can a *malicious tool*, a *Sybil* identity, or a *poisoned* entry in shared memory spread through the collective? These are not attributes of any one agent, and re-running a single-agent benchmark N times does not create them: it produces N independent agents, not a coordinated adversary, an orchestrator with a trust setting, or a compromise that propagates over a communication topology. The recent systematization of multi-agent security (MASEC) [61] makes the point sharply—security in multi-agent systems is *non-compositional*, in that individually safe agents can compose into unsafe systems—and states plainly, in its Section 4.7, that no multi-agent security benchmark exists, enumerating what one must provide: heterogeneous roles, partial observability, forced coordination, a co-evolving red/blue methodology, and group-level emergence metrics. A parallel survey of multi-agent risks from advanced AI [24] organizes the same territory into miscoordination, conflict, and collusion, again distinct from

single-agent mitigations. The field therefore cannot currently answer a basic question: *do single-agent defenses survive multi-agent deployment?*

Contribution. We answer the instrument gap directly. MASGYM is, to the best of our knowledge and within the verified literature, the first environment purpose-built for the security of multi-agent LLM systems. It is not a new attack and not a new defense; it is an environment and an evaluation methodology whose scientific object is the controlled variation of coordination, collusion, and orchestrator variables and the system-level metrics that quantify their effect. Figure 1 previews the design: a ToolEmu-style emulation layer supplies breadth of tools; an AgentDojo-style deterministic checker scores system-level security predicates over the global state trace, never placing an LLM judge on the critical path; and an orchestration layer instantiates topology, roles, partial observability, and a parameterized adversary. We make four concrete contributions and one empirical claim.

1. **An environment and task suite for multi-agent LLM security** (Sections 4 and 7). Heterogeneous roles (orchestrator, workers, tool-executors, shared-memory store), partial observability, and forced-coordination tasks, instantiated over star, chain, tree, and mesh topologies at 3–50 agents. Forced coordination, collusion, compromise propagation, and Sybil pressure are group-level *by construction* and have no single-agent analogue.
2. **A natively parameterized adversary model** (Section 5). A single configuration vector controls the adversary fraction over agents, independent versus colluding behaviour, static versus adaptive strategy, orchestrator compromise on/off, malicious tools, Sybil identities, and poisoned shared memory, with a system-level compromise objective (unsafe action, exfiltration, or coordination sabotage).
3. **A system-level metric suite** (Sections 6 and 7). A generalization $\text{NRP}_{\text{sys}} = \text{PNA}_{\text{sys}} \cdot (1 - \text{ASR}_{\text{sys}})$ of ASB’s metric, together with collusion success rate, compromise-propagation rate, coordination quality, time-to-detection, recovery rate, and cost, all measured over the collective rather than per step. Because ASR_{sys} is scored by a deterministic checker, a successful injection cannot hijack the evaluator—a property we prove and plan to quantify an LLM-judge ablation against (deferred to the release phase).
4. **The CoRB co-evolving red/blue protocol** (Section 8). A reusable methodology that alternates a red policy proposing attack configurations from the parameter space and a blue policy proposing defenses, logging a trajectory of (attack, defense) pairs and their system-level scores.
5. **A headline degradation result** (Section 10). We measure on a transparent, clearly-labelled synthetic agent emulator that defenses effective in the single-agent regime *degrade* under collusion and compromised-orchestrator conditions: a per-agent guardrail’s relative ASR reduction over no-defense shrinks $41.6\% \rightarrow 31.8\% \rightarrow 27.7\%$ across the three contexts (Table 3). Real-backbone replication is the release-phase programme; whether the effect replicates across models and topologies will confirm or reframe the result.

Because reviewers rightly discount benchmarks whose central metric is asserted rather than justified, and because the idea’s own risk analysis flags construct validity as the

primary threat, we treat the theory as part of the instrument rather than as decoration. Section 9 states, under explicit assumptions and with honest proof-status labels, five results: an axiomatic characterization that singles out the product aggregator among all utility–security combiners and reduces it to ASB’s metric when $N = 1$; an evaluator-integrity theorem showing that the deterministic checker’s verdict is invariant to adversarial message content and is therefore not hijackable, in contrast to an LLM judge; topology-dependent bounds on compromise propagation across the four topologies together with a strict collusion-versus-independent separation in the environment’s cascade model; a forced-coordination lower bound; and finite-sample concentration guarantees for the estimators that fix the number of episodes the sweeps require. None of these results asserts that any real defense is secure or that any defense provably degrades; they characterize the *measurement*, which is what a benchmark can honestly own.

Scope and stance. MASGYM is an instrument, not a verdict. A lower system-level attack success rate or a higher system-level NRP is a *measurement under a stated configuration*, not a certificate of security; we never equate the two. Emulation fidelity is the central threat to validity—ToolEmu’s reported human agreement is $\kappa \approx 0.48$ —and we surface it as a first-class caveat, route scoring through the deterministic checker, and report an emulated-versus-real-tools ablation rather than treating emulator verdicts as ground truth. Following the trusted computing base of the source design, we take the deterministic state checker and the environment harness to be trusted and give the emulated tools no real side effects; real-world tool side effects, physical actuation, and model-weight attacks are out of scope. The remainder of the paper is organized as follows. Section 2 situates MASGYM against single-agent benchmarks, multi-agent frameworks and attacks, and reinforcement-learning environments; Section 3 fixes notation and recalls the ASB, AgentDojo, and ToolEmu primitives; Sections 4 to 6 give the system model, threat model, and problem formulation; Sections 7 and 8 present the architecture, metric suite, and CORB protocol; Section 9 develops the theory; Section 10 specifies the evaluation protocol and the synthetic-model results; and Sections 11 to 14 discuss implications, limitations, ethics, and conclusions.

2 Related Work

We organize prior work into six clusters and then state precisely how MASGYM differs. Throughout, we distinguish *single-agent* instruments, which measure one agent against untrusted data, from the *multi-agent* regime that MASGYM targets.

2.1 Single-agent agent-security benchmarks and sandboxes

The closest instruments to MASGYM evaluate a lone tool-using agent. ASB [78] formalizes attacks (direct and indirect prompt injection, memory poisoning, a plan-of-thought backdoor, and mixed attacks) and eleven defenses across ten scenarios and many tools, and introduces the utility–security metric $\text{NRP} = \text{PNA} \cdot (1 - \text{ASR})$ that we generalize. Its threat model is single-agent with retrieval-augmented memory, and its sole cross-agent vector is a shared memory database. AgentDojo [14] contributes a dynamic, stateful environment whose *deterministic*, state-based `security()` and `utility()` functions are computed from pre- and post-execution environment state, so that a prompt injection

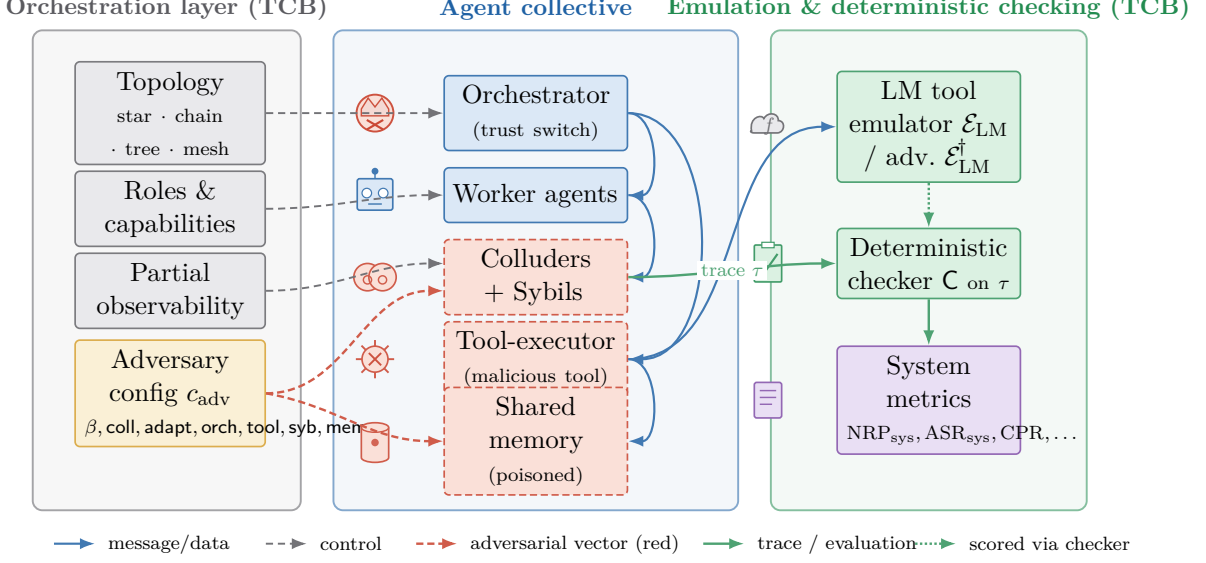


Figure 1: MASGYM architecture. The *orchestration layer* (left, trusted) sets the topology, roles, observability masks, and the adversary configuration c_{adv} (Equation (2)); the *agent collective* (centre) runs heterogeneous roles, with adversarial elements in red (a trust-switchable orchestrator, colluders and Sybils, a malicious tool, and poisoned shared memory); the *emulation and checking layer* (right, trusted) supplies emulated tools and scores system-level predicates Φ, Ψ over the global trace τ with a deterministic checker rather than an LLM judge, so a successful injection cannot hijack the score (Theorem 9.4). Colour and shape jointly encode role and adversarial status for grayscale readability.

which hijacks the agent cannot also hijack the evaluator; it explicitly names a multi-agent planner/worker defense as future work. InjecAgent [77] benchmarks indirect prompt injection in tool-integrated agents, and the ToolEmu sandbox [58] uses a language model to emulate tool execution from specifications, reporting that a majority of emulator-identified failures are valid real-world failures while cautioning that automatic evaluation agrees with humans only moderately. AgentHarm [1] measures the harmfulness of agent behaviour, HarmBench [46] standardizes automated red teaming and robust refusal, DecodingTrust [65] assesses trustworthiness dimensions of GPT models, and Liu et al. [44] formalize and benchmark prompt-injection attacks and defenses. All of these are single-agent; none parameterizes collusion, orchestrator trust, Sybils, or compromise propagation.

2.2 Multi-agent LLM frameworks and agent foundations

A large body of work builds orchestrated collectives of LLM agents: AutoGen [72], MetaGPT [29], CAMEL [39], ChatDev [55], AgentVerse [9], the dynamic agent network DyLAN [45], Mixture-of-Agents [66], multi-agent debate [16], ChatEval [8], and generalist systems such as OpenHands [69] and Magentic-One [18]. These rest on single-agent capabilities—ReAct-style reasoning and acting [74], Reflexion [62], tool use [60], and long-horizon agent behaviour [51]—surveyed by Guo et al. [23]. This literature is concerned with capability and coordination, not security; MASGYM adopts its role structures (planner/worker/tool/memory) as the substrate on which security is measured.

2.3 Attacks on multi-agent LLM systems

A fast-growing line studies attacks that are intrinsically multi-agent. Prompt Infection [36] shows LLM-to-LLM prompt injection propagating within a system; CORBA [82] induces contagious, recursive blocking; Agent Smith [22] jailbreaks a population of multimodal agents from a single seed at exponential speed; communication attacks place an adversary in the middle of inter-agent messages [26]; NetSafe [75] studies the topological safety of agent networks; PsySafe [79] attacks multi-agent safety through psychological manipulation; Evil Geniuses [64] probes the safety of agent roles; and zero-click GenAI worms [13] spread through agentic applications. Memory- and weight-level vectors include AgentPoison [10], which poisons an agent’s memory or knowledge base, and agent backdoors [73]. These works demonstrate that group-level vectors exist and are dangerous; they are attacks, evaluated in bespoke setups. MASGYM is complementary: it is the controlled environment in which such attacks—and the defenses meant to stop them—can be instantiated, swept, and compared under a common, deterministically scored metric.

2.4 Defenses, surveys, and systematizations

On the defensive side, AutoDefense [76] uses a multi-agent pipeline to filter jailbreaks, and G-Safeguard [68] adds a topology-guided detection-and-treatment lens to multi-agent systems. Single-agent guardrails such as TrustAgent [30], Llama Guard [31], and NeMo Guardrails [57] are the per-agent defenses we lift into the multi-agent setting as baselines. Classical Byzantine fault tolerance [35, 52] and Byzantine-robust aggregation such as Krum [3] tolerate a faulty fraction of nodes under a trusted coordinator and over numeric messages; they do not target natural-language agent collectives with a possibly compromised orchestrator, but they motivate MASGYM’s collusion and orchestrator-trust knobs. Surveys of agent security and privacy [25, 40] catalogue the single-agent attack surface, while MASEC [61] and the multi-agent risk taxonomy of Hammond et al. [24] argue that multi-agent security is non-compositional and enumerate the missing measurement apparatus. MASGYM is a direct response to that enumeration.

2.5 Prompt injection and jailbreak primitives

The attack primitives instantiated inside MASGYM originate in the single-agent literature: indirect prompt injection against LLM-integrated applications [21, 43], instruction-override techniques [54], gradient-based universal triggers [83], stealthy jailbreak generation [42], and analyses of why safety training fails [71]. MASGYM treats these as pluggable red-team primitives rather than as objects of study, and its contribution is the multi-agent *composition* of such primitives across roles, tools, memory, and identities.

2.6 Reinforcement-learning environments and co-evolving red teaming

Methodologically, MASGYM follows the tradition of standardized environments—the original Gym [5], the multi-agent PettingZoo [63], and Melting Pot [38], which evaluates social outcomes and generalization at the level of the collective—and of automated red teaming, in which one model generates test cases against another [53]. The MAST taxonomy of multi-agent failure modes [6] supplies a vocabulary for the group-level failures MASGYM is built to elicit. Our CORB protocol adapts the red/blue co-evolution idea to

the security of agent collectives and logs system-level scores rather than per-utterance verdicts.

2.7 Positioning: why MASGym is not a rescaled single-agent benchmark

Three distinctions matter. First, MASGYM is *not ASB applied to many agents*. ASB’s only cross-agent vector is shared memory, and it has no collusion and no compromised-orchestrator setting; re-running it N times yields N independent agents, never a coordinated adversary, an orchestrator with a trust switch, or a compromise that propagates over a topology. Second, MASGYM is *not AgentDojo at scale*. AgentDojo’s deterministic checker is a decisive design choice that MASGYM preserves and generalizes, but it was defined over a single agent’s environment state; scoring a *collective* requires system-level security predicates over a *global* trace, which is a new object (Section 6). Third, MASGYM is *not* a model-swap, a prompt-filter study, or mere engineering: its scientific object is the controlled variation of coordination, collusion, and orchestrator variables—undefined in a single-agent setting—and the system-level metrics that quantify their effect. The evaluator-integrity distinction is also load-bearing: LLM-as-judge evaluation is known to be manipulable and biased [67, 80], which is exactly why MASGYM keeps a deterministic checker on the critical path and treats the deterministic-versus-judge comparison as a measurable ablation rather than an assumption.

3 Preliminaries and Notation

This section fixes notation (Table 1) and recalls the three primitives MASGYM builds on: ASB’s utility–security metric, AgentDojo’s deterministic state-based checking, and ToolEmu’s LM tool emulation.

3.1 ASB’s utility–security metric

ASB [78] scores an agent by a *Non-Refusal Performance* that multiplies benign task performance by the complement of attack success,

$$\text{NRP} = \text{PNA} \cdot (1 - \text{ASR}), \quad (1)$$

where $\text{PNA} \in [0, 1]$ is performance on benign tasks (the agent neither refuses nor fails a legitimate request) and $\text{ASR} \in [0, 1]$ is the attack success rate. The product form rewards a system only when it is both useful and not successfully attacked; refusing everything drives PNA (hence NRP) to zero, and being fully compromised drives $\text{ASR} \rightarrow 1$ (hence $\text{NRP} \rightarrow 0$). Equation (1) is defined for a single agent; generalizing it to a collective is the subject of Sections 6 and 9.

3.2 AgentDojo’s deterministic state-based checking

AgentDojo [14] evaluates security by *deterministic* functions of environment state rather than by an LLM judge. Concretely, a task fixes a predicate that reads the pre- and post-execution state (e.g., “was money transferred to an account not named in the user’s instruction?”) and returns a Boolean. Because the predicate is a fixed program

Table 1: Notation used throughout. Dimensions and domains are stated where relevant; symbols are checked for conflicts in `notation_consistency_check.md`.

Symbol	Meaning
$\mathcal{A} = \{a_1, \dots, a_N\}$	set of N agents, $N \in \{3, \dots, 50\}$
$\mathcal{R}$	role set {orchestrator, worker, tool-executor, memory}
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	communication topology (star, chain, tree, mesh)
$\mathcal{N}^-(a)$	inbound neighbourhood of agent a in $\mathcal{G}$
$\mathcal{S}, s_t \in \mathcal{S}$	global state space; global state at step t
$\tau \in \mathcal{T}$	global state trace $\tau = (s_0, \dots, s_H)$ of an episode
H	episode horizon (number of environment steps)
$\Phi : \mathcal{T} \rightarrow \{0, 1\}$	system-level security predicate (1 = compromised)
$\Psi : \mathcal{T} \rightarrow \{0, 1\}$	benign-success predicate (1 = task solved)
C	deterministic checker computing $(\Psi(\tau), \Phi(\tau))$
$\mathcal{E}_{\text{LM}}$	LM tool emulator (ToolEmu-style); adversarial mode $\mathcal{E}_{\text{LM}}^\dagger$
J	LLM judge (contrast baseline for the evaluator)
$c \in \mathcal{C}$	environment configuration (topology, roles, adversary knobs)
$\mathcal{B} \subseteq \mathcal{A}$	set of adversarial agents; $\beta = \mathcal{B} /N$
$\text{ASR}_{\text{sys}}, \text{PNA}_{\text{sys}}, \text{NRP}_{\text{sys}}$	system-level attack success, benign performance, net metric
CPR, CSR	compromise-propagation rate, collusion success rate
$p \in [0, 1]$	per-edge compromise-propagation probability
$\theta \in \{1, 2, \dots\}$	compromise threshold (bootstrap-percolation model)
ρ, δ	red (attack) policy and blue (defense) policy in CoRB
M	number of evaluation episodes per configuration
κ	emulator/evaluator human-agreement coefficient (≈ 0.48)

over recorded state, a prompt injection that manipulates the agent’s natural-language reasoning cannot manipulate the score unless it actually changes the security-relevant state. MASGYM preserves this property and lifts it to the system level: our checker C evaluates predicates Φ, Ψ over the *global* trace τ , never an LLM judge on the critical path (Section 6), and we prove the resulting integrity property in Theorem 9.4 and quantify its value against an LLM-judge ablation in Section 10.

3.3 ToolEmu’s LM tool emulation

ToolEmu [58] replaces hand-built tool implementations with a language model that emulates tool execution from a specification, enabling breadth of tools and scenarios at low cost, and pairs the emulator with an automatic safety evaluator. Its authors validate both against humans and report only moderate agreement—human agreement of the automatic evaluation is $\kappa \approx 0.48$ (Cohen’s κ [12], an agreement coefficient extended to many raters by Fleiss [17]), and a majority (but not all) of emulator-identified failures are judged real. MASGYM adopts LM emulation for tool breadth, including an adversarial-emulator mode $\mathcal{E}_{\text{LM}}^\dagger$ for malicious tools, but treats κ as a first-class threat to validity: scoring is routed through the deterministic checker (not the emulator’s own verdict), and an emulated-versus-real-tools ablation is part of the protocol (Section 10).

3.4 Episodes, traces, and configurations

An *episode* runs a task to completion under a fixed configuration c and produces a global state trace τ . The configuration c specifies the topology $\mathcal{G}$, the role assignment, the observability mask, and the adversary knobs of Section 5. We write $\tau \sim \mathcal{D}(c)$ for the distribution over traces induced by running the (stochastic) agents and environment under c , and reserve $\mathbb{K}[\cdot]$ for the indicator function. All security and utility quantities are functionals of τ and are therefore, by construction, computable by a program that reads only the recorded trace—the fact that makes deterministic, hijack-resistant scoring possible.

4 System Model

MASGYM instantiates an orchestrated collective of LLM agents. Because it is an environment, the model below is *configurable*: the quantities that a security study wants to vary—topology, roles, observability, trust—are parameters of the harness, not fixed choices.

4.1 Agents and roles

The system comprises $N \in \{3, \dots, 50\}$ agents $\mathcal{A} = \{a_1, \dots, a_N\}$, each an LLM policy that reads a local context and emits messages and tool calls. Agents are *heterogeneous* in role, drawn from $\mathcal{R}$:

- an **orchestrator** (planner) that decomposes the task, assigns subtasks, and routes messages;
- **workers** that solve subtasks and exchange intermediate results;
- **tool-executors** that mediate calls to external tools; and
- a **shared-memory store** that agents read from and write to.

Each agent a has capabilities: a model backbone (Section 10), a set of permitted tools, and read/write permissions on shared memory. The role assignment and capability grants are part of the configuration c .

4.2 Topology and communication

Agents communicate over a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with $\mathcal{V} = \mathcal{A}$; an edge $(a, a') \in \mathcal{E}$ means a' may read messages from a . MASGYM ships four canonical topologies (Figure 1): a **star** with the orchestrator at the hub; a **chain** in which each agent forwards to its successor; a **tree** in which the orchestrator branches to sub-orchestrators and workers; and a **mesh** in which many worker pairs communicate directly. Topology is a first-class variable because, as Section 9 shows, it controls how a compromise propagates and how severe an orchestrator compromise is.

4.3 Partial observability

Agents are *partially observing*: agent a sees only its local context—its own role prompt, the messages on its inbound edges $\mathcal{N}^-(a)$, and the memory entries it is permitted to read. No agent observes the global state s_t ; in particular, no honest agent can unilaterally detect that a peer has been compromised or that the orchestrator is malicious. Partial observability is what makes group-level detection nontrivial and is a stated requirement of the multi-agent security agenda [61].

4.4 Forced coordination

A task is *forced-coordination* when its benign-success predicate Ψ can be satisfied only by the joint action of several agents—no single agent, and no coalition below a task-specific size, can complete it alone (Theorem 6.3 makes this precise). Forced coordination is the property that makes the collective the unit of analysis: it cannot be represented in a single-agent harness, and it lets MASGYM measure coordination quality and coordination sabotage as system-level quantities.

4.5 Orchestration, trust boundaries, and the harness

An **orchestration layer**—part of the trusted harness—instantiates $\mathcal{G}$, assigns roles, enforces the observability masks, applies the adversary knobs, and drives episodes step by step. Per-agent **trust boundaries** record which agents and tools a given agent is configured to trust; the orchestrator itself carries a trust bit that can be set to *trusted* or *compromised* (Section 5). Optional **human-approval hooks** can gate designated high-risk actions, letting a study measure the human-intervention rate required to hold risk at a target level.

4.6 Trusted computing base

Following the source design, the **deterministic state checker** $\mathbf{C}$ and the **environment harness** (orchestration layer, logging, observability enforcement) are the trusted computing base (TCB) and are assumed uncompromised; the emulated tools have *no real side effects*. Everything else—agents (including the orchestrator), tools, memory contents, and messages—may be adversarial under a given configuration. Out of scope are real-world tool side effects, physical actuation, and model-weight attacks. This TCB is what makes hijack-resistant scoring possible: because $\mathbf{C}$ reads only the harness-recorded trace τ , no adversarial agent or tool can alter a verdict except by actually changing the security-relevant state that the harness records (Theorem 9.4).

5 Threat Model

Because MASGYM is an environment, its threat model is *parameterized and configurable*, not fixed. A study selects a point in the adversary parameter space, and the harness realizes it. Figure 2 depicts the resulting honest, adversarial, and orchestrator/tool/memory vectors and how each enters the system.

5.1 Adversary configuration

An adversary is specified by a configuration

$$c_{\text{adv}} = (\beta, \text{coll}, \text{adapt}, \text{orch}, \text{tool}, \text{syb}, \text{mem}), \quad (2)$$

whose components the harness exposes as independent knobs:

- **Adversary fraction** $\beta = |\mathcal{B}|/N \in [0, 1]$: the fraction of agents placed under adversarial control, $\mathcal{B} \subseteq \mathcal{A}$.
- **Collusion** $\text{coll} \in \{\text{independent}, \text{colluding}\}$: whether adversarial agents optimize a shared objective and may coordinate messages, timing, and target selection, or act independently.
- **Adaptivity** $\text{adapt} \in \{\text{static}, \text{adaptive}\}$: whether the red strategy is fixed in advance or may react to the blue defense across CORB rounds (Section 8).
- **Orchestrator compromise** $\text{orch} \in \{\text{trusted}, \text{compromised}\}$: a single bit flipping the planner from honest to adversarial. A compromised orchestrator can mis-route, mislabel, or fabricate subtasks and messages.
- **Malicious tools** tool : a subset of tools whose (emulated) behaviour is adversarial, realized by the adversarial emulator $\mathcal{E}_{\text{LM}}^\dagger$ (e.g., returning attacker-controlled content or exfiltration bait).
- **Sybil identities** syb : the number of additional identities an adversary may instantiate to gain disproportionate influence over selection, voting, or memory writes (cf. the classical Sybil attack [15]).
- **Poisoned memory** mem : adversarial entries written into the shared-memory store, in the spirit of memory-poisoning attacks on single agents [10] but now readable by many agents.

5.2 Attacker objective, knowledge, and capabilities

Objective. The adversary seeks *system-level compromise*: causing an unsafe action, exfiltrating protected information, or sabotaging coordination so the collective fails a forced-coordination task. Formally, the adversary maximizes $\mathbb{P}[\Phi(\tau) = 1]$ (attack success) and, for sabotage objectives, minimizes $\mathbb{P}[\Psi(\tau) = 1]$ (benign success), where Φ, Ψ are the fixed predicates of Section 6.

Knowledge. MASGYM supports a spectrum. At minimum the adversary knows its own role, tools, and local context; at maximum (a strong-adversary study) it knows the topology $\mathcal{G}$, the honest agents' prompts, and the active blue defense. Knowledge is a configurable dimension so that a defense is not credited for security that evaporates under a better-informed attacker.

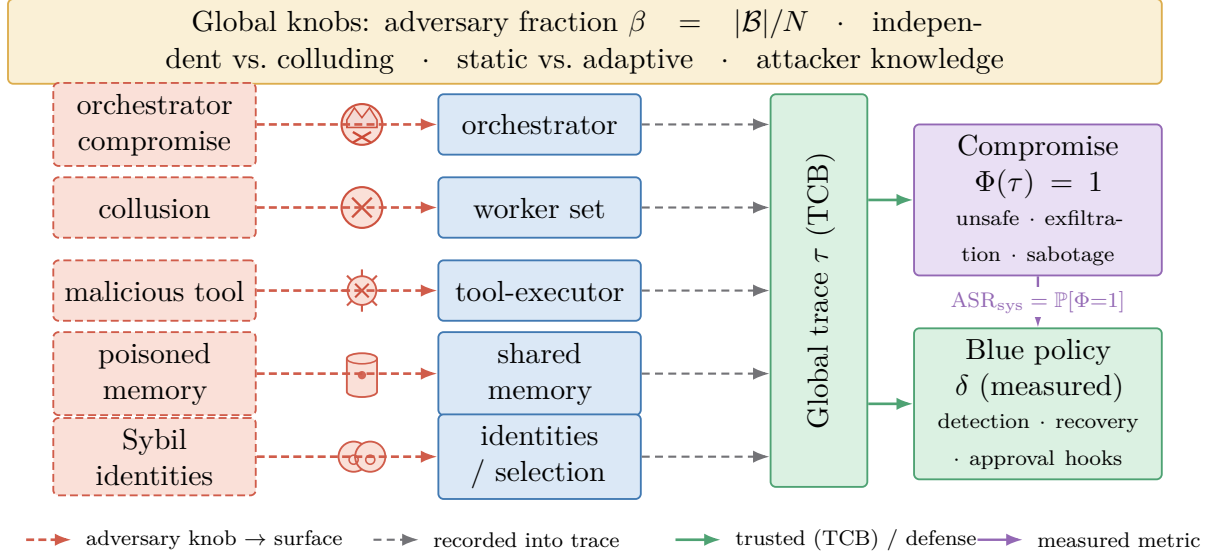


Figure 2: Parameterized threat model. Global knobs (top) set the adversary fraction, collusion, adaptivity, and knowledge; per-vector knobs (left, red) each enter a specific attack surface (a trust-switchable orchestrator, a colluding worker set, a malicious tool, poisoned shared memory, and Sybil identities). Every action is recorded by the trusted harness into the global trace τ , over which the deterministic checker evaluates the compromise predicate Φ ; the blue policy’s detection and recovery are *measured*, never assumed. A low ASR_{sys} is a measurement under a stated configuration, not a certificate of security.

Capabilities. An adversarial agent may emit arbitrary messages and tool calls within its granted permissions, write to memory it can write to, and—if colluding—share state with other adversarial agents out of band. A compromised orchestrator additionally controls routing and task assignment. Adversaries *cannot* read observations they are not permitted to see, forge messages on edges they do not control (the harness attributes messages to their senders), or alter the recorded trace τ ; these are TCB guarantees.

5.3 Static versus adaptive, independent versus colluding

The two axes that distinguish the multi-agent regime are collusion and adaptivity. An *independent* adversary of fraction β behaves like βN separately acting bad agents; a *colluding* adversary of the same fraction coordinates seed placement, message content, and timing toward the shared objective. A *static* adversary commits to a strategy; an *adaptive* adversary co-evolves against the blue policy. Section 9 shows that in the environment’s propagation model, collusion yields a strict advantage on star and tree topologies; Section 10 tests whether that advantage manifests as measured defense degradation for real defenses (a *hypothesis*).

5.4 Trust assumptions and scope

Trusted: the deterministic checker C and the harness (Section 4.6). Untrusted under the appropriate knob: any agent (including the orchestrator), any tool, any memory entry, and any message content. Out of scope: real-world tool side effects, physical actuation, and model-weight or training-time attacks on the backbones. We emphasize the stance of the whole paper: a configuration that yields low ASR_{sys} or high NRP_{sys} demonstrates *measured* risk under the stated knobs; it is never read as a proof of security.

6 Problem Formulation

We now define the objects a system-level security benchmark must measure: the security and benign-success predicates, the system-level metric that aggregates them, and the group-level quantities (forced coordination, compromise propagation, collusion) that have no single-agent analogue. These definitions are the contract the theory of Section 9 must satisfy and the environment of Section 7 must realize.

6.1 System-level predicates

Definition 6.1 (System-level security and benign-success predicates). A task fixes two Boolean functionals of the global trace $\tau \in \mathcal{T}$: a *security predicate* $\Phi : \mathcal{T} \rightarrow \{0, 1\}$ with $\Phi(\tau) = 1$ iff the trace contains a system-level compromise (an unsafe action executed, protected data exfiltrated, or a forced-coordination task sabotaged), and a *benign-success predicate* $\Psi : \mathcal{T} \rightarrow \{0, 1\}$ with $\Psi(\tau) = 1$ iff the collective completed the legitimate task. Both are fixed programs registered before evaluation and are evaluated by the deterministic checker $\mathbf{C}$ over the harness-recorded trace.

The requirement that Φ, Ψ depend on τ *only*—and never on un-recorded natural-language content—is what later yields evaluator integrity (Theorem 9.4). It also mirrors AgentDojo’s single-agent design [14], lifted from one agent’s environment state to the global trace of the collective.

6.2 System-level metrics

Definition 6.2 (System-level ASR, PNA, and NRP). Fix a configuration c and let $\tau \sim \mathcal{D}(c)$. Define

$$\text{ASR}_{\text{sys}}(c) = \mathbb{P}_{\tau \sim \mathcal{D}(c)}[\Phi(\tau) = 1], \quad (3)$$

$$\text{PNA}_{\text{sys}}(c) = \mathbb{P}_{\tau \sim \mathcal{D}(c^\circ)}[\Psi(\tau) = 1], \quad (4)$$

$$\text{NRP}_{\text{sys}}(c) = \text{PNA}_{\text{sys}}(c) \cdot (1 - \text{ASR}_{\text{sys}}(c)), \quad (5)$$

where c° denotes the benign counterpart of c (no adversary), so that PNA_{sys} measures coordinated-task performance uncontaminated by refusals induced by the attack, exactly as ASB separates PNA from ASR.

Two properties are required and proved in Section 9: Equation (5) must *reduce* to ASB’s Equation (1) when $N = 1$ and Φ is the single-agent predicate (Theorem 9.1), and the product form must be *justified* rather than assumed—we give an axiomatic characterization that singles it out (Theorem 9.2). We stress the benchmark stance: NRP_{sys} is a coordinate for comparing configurations, not a certificate; “higher NRP_{sys} ” means “measured to be more useful-and-less-compromised under this configuration,” nothing more.

6.3 Forced coordination

Definition 6.3 (k -forced-coordination task). For an agent set $\mathcal{A}$, let a *witness* be a set $W \subseteq \mathcal{A}$ of agents whose joint contributions can satisfy Ψ . A task is *k -forced-coordination* if every witness has size $|W| \geq k$; equivalently, for every set S with $|S| < k$, no assignment in which only the agents of S act non-trivially satisfies Ψ .

k -forced tasks with $k \geq 2$ cannot be represented in a single-agent harness and make the collective the unit of analysis. Section 9 relates task disruption to whether the adversary set $\mathcal{B}$ intersects every minimal witness (a hitting-set condition).

6.4 Compromise propagation and collusion

Definition 6.4 (Compromise relation and propagation rate). Let $\mathcal{B}_0 = \mathcal{B}$ be the initially adversarial (seed) agents. Compromise spreads over $\mathcal{G}$ according to a monotone rule $\mathcal{P}$ (Section 9 instantiates $\mathcal{P}$ as an independent-cascade model with per-edge probability p and, alternatively, a threshold model with threshold θ). Write $\mathcal{B}_\infty \supseteq \mathcal{B}_0$ for the set compromised at fixation. The *compromise-propagation rate* is

$$\text{CPR}(c) = \mathbb{E} \left[\frac{|\mathcal{B}_\infty| - |\mathcal{B}_0|}{N - |\mathcal{B}_0|} \right], \quad (6)$$

the expected fraction of *initially honest* agents that become compromised.

Definition 6.5 (Collusion success rate). For a fixed fraction β , let $\text{ASR}_{\text{sys}}^{\text{coll}}$ and $\text{ASR}_{\text{sys}}^{\text{ind}}$ be the system attack success rates under colluding and independent adversaries respectively. The *collusion success rate* reports $\text{ASR}_{\text{sys}}^{\text{coll}}$, and the *collusion advantage* is $\Delta_{\text{coll}} = \text{ASR}_{\text{sys}}^{\text{coll}} - \text{ASR}_{\text{sys}}^{\text{ind}}$.

6.5 Desiderata for the aggregator

Finally, we record the properties a system-level utility–security aggregator $g : [0, 1] \times [0, 1] \rightarrow [0, 1]$, written $\text{NRP}_{\text{sys}} = g(\text{PNA}_{\text{sys}}, 1 - \text{ASR}_{\text{sys}})$, should satisfy. These are the axioms discharged in Theorem 9.2: (D1) *boundedness*, g maps into $[0, 1]$; (D2) *security consistency*, $g(u, 1) = u$ (no attack means net performance equals utility) and $g(0, \sigma) = 0$ (no utility means no net performance); (D3) *monotonicity*, g is non-decreasing in each argument; (D4) *utility- and security-proportionality*, the two scaling axioms of Theorem 9.2; and (D5) *reduction*, g recovers Equation (1) at $N = 1$. The point of stating D1–D5 is that the central metric of the benchmark is then a theorem, not a stipulation—directly answering the construct-validity concern that motivates this work.

7 The MASGym Environment

MASGYM is organized as three layers (Figures 1 and 3): an *emulation layer* that supplies tools, a *deterministic checking layer* that scores system-level predicates, and an *orchestration layer* that instantiates the system and threat models and drives the CORB loop. We describe each layer, the task suite, the realized adversary knobs, and the full metric suite.

7.1 Emulation layer

Following ToolEmu [58], tool execution is emulated by a language model $\mathcal{E}_{\text{LM}}$ from a tool specification, giving breadth without hand-built sandboxes. MASGYM adds an *adversarial-emulator* mode $\mathcal{E}_{\text{LM}}^\dagger$ that realizes malicious tools: under the tool knob, a designated tool’s emulated responses are generated to carry attacker-controlled content, exfiltration bait, or injected instructions. Crucially, the emulator is *not* on the scoring

critical path: it produces tool outputs that agents consume, but the security verdict is computed by the deterministic checker from the recorded trace, so emulator noise (the $\kappa \approx 0.48$ caveat) degrades realism, not scoring integrity. An emulated-versus-real-tools ablation (Section 10) bounds the realism cost.

7.2 Deterministic checking layer

The checker C evaluates the registered predicates Φ, Ψ over the global trace τ (Theorem 6.1). It never invokes an LLM judge on the critical path. System-level predicates are expressed as programs over the recorded state: for example, an *exfiltration* predicate fires iff protected data recorded as leaving the trust boundary appears in an outbound tool call; a *sabotage* predicate fires iff a k -forced task’s witness set was prevented from completing; an *unsafe-action* predicate fires iff a state transition on a denylisted action is recorded. Because these are functions of τ alone, they inherit the hijack-resistance property proved in Theorem 9.4. For studies that wish to *measure* the risk of judge-based evaluation, MASGYM also ships an optional LLM judge J used only off the critical path, enabling the deterministic-versus-judge ablation.

7.3 Orchestration layer

The orchestration layer builds the topology $\mathcal{G}$, assigns roles, enforces observability masks, applies the adversary configuration c_{adv} of Equation (2), and runs episodes. It is part of the TCB (Section 4.6): it attributes every message to its sender, records the trace, and gates human-approval hooks. The same layer exposes the CORB interface, alternately querying a red policy for an attack configuration and a blue policy for a defense, and logging their system-level scores (Section 8).

7.4 Task suite

MASGYM tasks are *forced-coordination* by design (Theorem 6.3), so that no single agent can complete them and the collective is the unit of analysis. Each task provides: a benign goal decomposable across roles; a registered pair (Ψ, Φ) ; a tool set (some emulated as malicious under the tool knob); and a shared-memory schema (poisonable under **mem**). Tasks span the coordination structures that the topologies induce—hub-and-spoke delegation (star), pipelines (chain), hierarchical decomposition (tree), and peer collaboration (mesh)—and are parameterized by k (the forced-coordination degree) and by benign difficulty so that PNA_{sys} is neither saturated nor zero.

7.5 Realized adversary knobs

Each component of c_{adv} maps to a concrete harness mechanism: β selects the seed set $\mathcal{B}$; **coll** toggles a shared adversary channel and a joint objective; **adapt** enables red co-evolution across CORB rounds; **orch** flips the orchestrator’s policy to adversarial; **tool** routes designated tools through $\mathcal{E}_{\text{LM}}^\dagger$; **syb** instantiates extra identities that share an underlying controller; and **mem** writes poisoned entries into shared memory. Because the knobs are independent, a study can isolate the effect of a single variable while holding the rest fixed—the controlled variation that is the paper’s scientific object, read as estimating the causal effect of a treatment such as collusion [59].

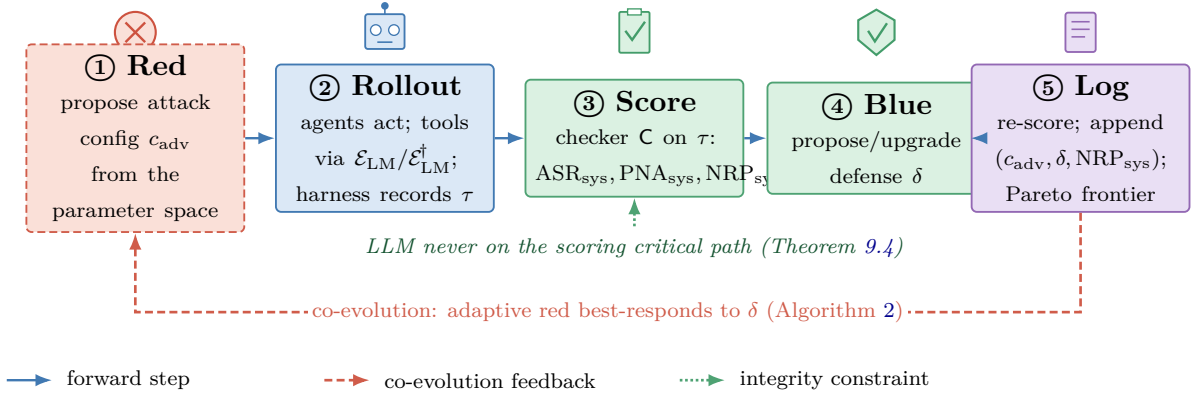


Figure 3: The CORB co-evolving red/blue protocol (Algorithm 2). A red policy proposes an attack configuration; the harness rolls out episodes and records the global trace τ ; the deterministic checker scores system-level metrics; a blue policy proposes or upgrades a defense; and the pair is re-scored and appended to the trajectory, from which a (security, cost) Pareto frontier is read. The adaptive setting closes the loop (red best-responds to the current blue). Because scoring is deterministic and off the LLM critical path, no proposal can inflate its own evaluation.

7.6 Metric suite

MASGYM reports the following system-level quantities, all computed from traces by the deterministic checker or the harness logs; none is an LLM-judge verdict on the critical path.

- **System attack success rate** ASR_{sys} (Equation (3)) and **system NRP** NRP_{sys} (Equation (5)); benign performance PNA_{sys} (Equation (4)).
- **Collusion success rate** CSR and collusion advantage Δ_{coll} (Theorem 6.5).
- **Compromise-propagation rate** CPR (Equation (6)): fraction of initially honest agents that become compromised.
- **Coordination quality**: benign success on forced-coordination tasks, and its degradation under sabotage objectives.
- **Time-to-detection**: environment steps until a defense (if any) flags the compromise, and **recovery rate**: fraction of episodes that return to a safe, task-completing state after detection.
- **Cost and scalability**: token/API cost, wall-clock latency, and how each metric and cost scale with the agent count N .
- **Human-intervention rate**: fraction of episodes in which a human-approval hook was required to hold risk at target, when such hooks are enabled.

Estimating these to a target precision requires a number of episodes fixed by the concentration results of Theorem 9.17, which in turn drives the cost analysis of Section 10.

Algorithm 1 SCORECONFIG: deterministic system-level scoring of one configuration.

- 1: **Input:** configuration c ; task with predicates (Ψ, Φ) ; episode budget M ; horizon H .
 - 2: **Output:** estimates $\widehat{\text{ASR}}_{\text{sys}}, \widehat{\text{PNA}}_{\text{sys}}, \widehat{\text{NRP}}_{\text{sys}}, \widehat{\text{CPR}}$ and detection/recovery statistics.
 - 3: Instantiate topology $\mathcal{G}$, roles, observability masks, and adversary knobs c_{adv} via the trusted harness. $\triangleright$ Sec. 7
 - 4: $\mathcal{B}_0 \leftarrow$ seed set of size $\lceil \beta N \rceil$; enable collusion channel iff $\text{coll} = \text{colluding}$.
 - 5: **for** $m = 1, \dots, M$ **do**
 - 6: $\tau^{(m)} \leftarrow ()$; reset global state s_0 .
 - 7: **for** $t = 0, \dots, H - 1$ **do**
 - 8: Each agent a acts on its local view; tool calls routed to $\mathcal{E}_{\text{LM}}$ or $\mathcal{E}_{\text{LM}}^\dagger$; memory reads/writes applied. $\triangleright$ malicious tools / poisoned memory
 - 9: Harness records the transition into $\tau^{(m)}$ and updates the compromise set $\mathcal{B}_t$ under rule $\mathcal{P}$. $\triangleright$ Def. 6.4
 - 10: $(u^{(m)}, v^{(m)}) \leftarrow (\Psi(\tau^{(m)}), \Phi(\tau^{(m)}))$ via $\mathcal{C}$. $\triangleright$ deterministic; never an LLM judge
 - 11: $\widehat{\text{ASR}}_{\text{sys}} \leftarrow \frac{1}{M} \sum_m v^{(m)}$; $\widehat{\text{PNA}}_{\text{sys}} \leftarrow \frac{1}{M} \sum_m u^{(m)}$ on the benign counterpart c° .
 - 12: $\widehat{\text{NRP}}_{\text{sys}} \leftarrow \widehat{\text{PNA}}_{\text{sys}} (1 - \widehat{\text{ASR}}_{\text{sys}})$; $\widehat{\text{CPR}} \leftarrow \frac{1}{M} \sum_m \frac{|\mathcal{B}_H^{(m)}| - |\mathcal{B}_0|}{N - |\mathcal{B}_0|}$.
 - 13: **return** the estimates and logged detection/recovery statistics.
-

8 The CoRB Co-Evolving Red/Blue Protocol

MASGYM’s evaluation methodology is a co-evolving loop, CoRB, that alternates a red policy proposing attack configurations and a blue policy proposing defenses, scoring each pair with the deterministic checker and logging a trajectory of (attack, defense, NRP_{sys}) triples. Algorithm 1 specifies how a single configuration is turned into system-level scores; Algorithm 2 specifies the co-evolution. Both keep the LLM off the scoring critical path, so no proposal can inflate its own score.

Design notes. Three properties make CoRB a measurement protocol rather than a training loop. (i) *Integrity*: every score in $\mathcal{L}$ is produced by Algorithm 1’s deterministic checker, so neither the red nor the blue policy can hijack its own evaluation (Theorem 9.4). (ii) *Reproducibility*: $\mathcal{L}$ is the full audit trail—each row is a fully specified (c_{adv}, δ) pair with its system-level scores and cost—so a result is a point in a logged trajectory, not a single headline number. (iii) *Honesty of the frontier*: CoRB reports a Pareto frontier over (security, cost) and (utility, cost) rather than a single “winner,” because a lower ASR_{sys} bought with unacceptable cost or utility loss is not a security success. The red and blue policies may be simple search (grid/random over the parameter space), an optimizer, or an LLM-driven proposer; MASGYM fixes the *interface* and the *scoring*, not the proposers, so the protocol is reusable as other red/blue policies are developed.

8.1 Complexity analysis

We account for the cost of the instrument along four axes: agent invocations (the dominant API/token driver), inter-agent messages (communication), trace storage (memory), and the deterministic checking overhead. Table 2 summarizes the asymptotics.

Algorithm 2 CoRB: co-evolving red/blue evaluation.

- 1: **Input:** red policy ρ over attack space; blue policy δ over defense space; rounds R ; task and budget M .
 - 2: **Output:** trajectory $\mathcal{L} = \{(c_{\text{adv}}^r, \delta^r, \text{NRP}_{\text{sys}}^r, \text{ASR}_{\text{sys}}^r)\}_{r=1}^R$ and Pareto frontier.
 - 3: Initialize defense δ^0 (e.g., no-defense or a lifted per-agent defense).
 - 4: **for** $r = 1, \dots, R$ **do**
 - 5: *Red step:* $c_{\text{adv}}^r \leftarrow \rho(\text{history } \mathcal{L}_{<r}, \delta^{r-1})$ $\triangleright$ static: fixed; adaptive: best-response
 - 6: Score $(c_{\text{adv}}^r, \delta^{r-1})$ with SCORECONFIG (Algorithm 1).
 - 7: *Blue step:* $\delta^r \leftarrow \delta(\text{history } \mathcal{L}_{\leq r}, c_{\text{adv}}^r)$ $\triangleright$ propose/upgrade defense
 - 8: Re-score $(c_{\text{adv}}^r, \delta^r)$ with SCORECONFIG; append both scorings to $\mathcal{L}$.
 - 9: **return** $\mathcal{L}$ and its $(\text{ASR}_{\text{sys}}, \text{cost}) / (\text{NRP}_{\text{sys}}, \text{cost})$ Pareto frontier.
-

Table 2: Cost of the MASGYM instrument. N agents, horizon H , topology edges $|\mathcal{E}|$ ($\Theta(N)$ for star/chain/tree, $\Theta(N^2)$ for dense mesh), M episodes per configuration, R CoRB rounds, G swept configurations. Checking is deterministic (no LLM calls).

Scope	Agent invocations	Trace storage / checking
Per episode	$\Theta(NH)$	$\Theta((N + \mathcal{E})H)$
Per configuration	$\Theta(MNH)$	$\Theta(M(N + \mathcal{E})H)$
Per CoRB run	$\Theta(RMNH)$	$\Theta(RM(N + \mathcal{E})H)$
Full sweep	$\Theta(GNH\Delta^{-2}\ln\frac{G}{\alpha})$	$\Theta(G(N + \mathcal{E})H \cdot M)$

Per episode. An episode runs N agents for H steps, so it makes $\Theta(NH)$ agent invocations and, on a topology with $|\mathcal{E}|$ edges, up to $\Theta(|\mathcal{E}|H)$ messages; the harness stores a trace of size $\Theta((N + |\mathcal{E}|)H)$. The checker evaluates the predicates Φ, Ψ once over the trace; for the predicate families we use (denylisted-action, exfiltration, and sabotage checks), each is a linear or near-linear scan of the trace, costing $\tilde{O}((N + |\mathcal{E}|)H)$ and never an LLM call. Edge counts differ by topology: $|\mathcal{E}| = \Theta(N)$ for star, chain, and tree, and $|\mathcal{E}| = \Theta(N^2)$ for a dense mesh, which is why the mesh is the costliest topology (Figure 6(d)).

Per configuration and per sweep. Estimating the metrics for one configuration to accuracy ϵ costs $M = O(\epsilon^{-2} \ln(1/\alpha))$ episodes (Theorem 9.17), hence $\Theta(MNH)$ invocations. A CoRB run of R rounds costs $2M$ scorings per round, i.e., $\Theta(RMNH)$ invocations. Sweeping G configurations to resolve NRP_{sys} gaps of size Δ costs $\Theta(GNH\Delta^{-2}\ln(G/\alpha))$ invocations by Theorem 9.18. The dominant practical driver is the product $G \cdot N$: both the grid size and the agent count. Because the full parameter space is exponential in the number of knobs of c_{adv} , CoRB explores a *reachable region* rather than the whole space—an honesty constraint we make explicit rather than hide, and which we report per study.

9 Theory: What the Instrument Measures

A benchmark earns trust when its central metric is justified, its evaluator cannot be gamed, and its sample sizes are principled. We therefore treat theory as part of the instrument. All results below are stated under the explicit assumptions of Section 9.1 and carry honest proof-status labels; none claims that any real defense is secure or provably degrades. Long

derivations are deferred to Section A. Figure 4 shows the dependency structure.

9.1 Assumptions

Assumption 1 (Trusted checking). The harness records a faithful global trace τ , and the checker C evaluates the fixed predicates Φ, Ψ as functions of τ *only* (Section 4.6).

Assumption 2 (i.i.d. episodes). For a fixed configuration c , the M episode traces are independent draws from $\mathcal{D}(c)$.

Assumption 3 (Bounded indicators). $\Psi, \Phi \in \{0, 1\}$, so per-episode utility and security indicators lie in $\{0, 1\}$ and their means in $[0, 1]$.

Assumption 4 (Monotone propagation model). Compromise spreads over $\mathcal{G}$ by a monotone (absorbing) rule $\mathcal{P}$. In the *independent-cascade* instance, a compromised agent independently compromises each susceptible out-neighbour with probability p when its message is processed; in the *threshold* instance, a susceptible agent is compromised once at least θ of its in-neighbours are compromised.

Assumption 5 (Continuity of the aggregator). The utility–security aggregator $g : [0, 1]^2 \rightarrow [0, 1]$ is continuous.

Assumption 4 defines the environment’s compromise dynamics; it is a modelling choice MASGYM implements so that propagation is measurable, not an empirical law about LLM agents. Its realism is itself something MASGYM measures (via CPR), and the emulated-versus-real-tools ablation.

9.2 Characterization of the system-level metric

We first show that the product form of NRP_{sys} is forced by the desiderata of Section 6, and that it reduces to ASB’s metric at one agent.

Proposition 9.1 (Reduction to ASB). *If $N = 1$ and Φ, Ψ are the single-agent security and benign predicates, then $\text{ASR}_{\text{sys}} = \text{ASR}$, $\text{PNA}_{\text{sys}} = \text{PNA}$, and $\text{NRP}_{\text{sys}} = \text{NRP}$ of Equation (1).*

Proof. With one agent the global trace is that agent’s trajectory and the system predicates equal the single-agent predicates, so Equations (3) and (4) equal ASB’s ASR and PNA, and Equation (5) equals Equation (1). $\square$

Theorem 9.2 (Axiomatic characterization of NRP_{sys} ; complete proof). *Write $\text{NRP}_{\text{sys}} = g(u, \sigma)$ with utility $u = \text{PNA}_{\text{sys}} \in [0, 1]$ and security level $\sigma = 1 - \text{ASR}_{\text{sys}} \in [0, 1]$. Under Assumption 5, suppose g satisfies: (D2) consistency, $g(u, 1) = u$ and $g(0, \sigma) = 0$; (D4a) utility-homogeneity, $g(\lambda u, \sigma) = \lambda g(u, \sigma)$ for $\lambda \in [0, 1]$; and (D4b) security-homogeneity, $g(u, \lambda \sigma) = \lambda g(u, \sigma)$ for $\lambda \in [0, 1]$. Then $g(u, \sigma) = u \sigma$ is the unique such aggregator, and it also satisfies boundedness (D1) and monotonicity (D3).*

Proof. By D4a with base $u = 1$, $g(u, \sigma) = g(u \cdot 1, \sigma) = u g(1, \sigma)$. By D4b with base $\sigma = 1$, $g(1, \sigma) = g(1, \sigma \cdot 1) = \sigma g(1, 1)$. By D2, $g(1, 1) = 1$. Hence $g(u, \sigma) = u \sigma g(1, 1) = u \sigma$. The boundary $g(0, \sigma) = 0$ holds since $0 \cdot \sigma = 0$. Finally $u \sigma$ is bounded in $[0, 1]$ and non-decreasing in each argument on $[0, 1]^2$. $\square$

Remark 9.3 (Construct validity, honestly). Theorem 9.2 says the product is the unique aggregator *relative to* $D2/D4$; it is not a claim that the product is “optimal” in any absolute sense. Replacing $D4$ by a worst-coordinate priority axiom yields $\min(u, \sigma)$; replacing it by weighted log-linearity yields a weighted geometric mean $u^w \sigma^{1-w}$. MASGYM reports $\text{NRP}_{\text{sys}} = u\sigma$ as its default because it generalizes ASB (Theorem 9.1) and matches the “useful-and-not-compromised” reading, and it exposes the alternatives as configurable so that construct validity is testable rather than asserted—exactly the concern that motivates this work.

9.3 Evaluator integrity

We now formalize why deterministic checking cannot be hijacked by a successful injection, and contrast it with an LLM judge.

Theorem 9.4 (Evaluator integrity; complete proof). *Under Assumption 1, for any two episodes with equal recorded traces $\tau = \tau'$, $\mathbf{C}(\tau) = \mathbf{C}(\tau')$. Consequently, if an adversarial manipulation changes message content but leaves the recorded security-relevant state unchanged (so the induced trace is unchanged), it cannot change $\Phi(\tau)$; hence neither $\widehat{\text{ASR}}_{\text{sys}}$ nor $\widehat{\text{NRP}}_{\text{sys}}$ can be inflated or deflated by message content alone.*

Proof. By Assumption 1, Φ and Ψ are functions of τ only; a function returns equal outputs on equal inputs, giving $\mathbf{C}(\tau) = \mathbf{C}(\tau')$. Let π be any message-space manipulation inducing trace τ_π . The verdict is $\Phi(\tau_\pi)$; if $\tau_\pi = \tau$, then $\Phi(\tau_\pi) = \Phi(\tau)$. As $\widehat{\text{ASR}}_{\text{sys}}$ and $\widehat{\text{PNA}}_{\text{sys}}$ are averages of such verdicts, they are unchanged, and so is $\widehat{\text{NRP}}_{\text{sys}}$. $\square$

Proposition 9.5 (An LLM judge is hijackable; complete proof of existence). *There exist an LLM judge $\mathbf{J}$ that reads messages and a message manipulation π with $\tau_\pi = \tau$ (no security-relevant state change) such that $\mathbf{J}(\tau_\pi, \text{msgs}_\pi) \neq \mathbf{J}(\tau, \text{msgs})$.*

Proof. Because $\mathbf{J}$ takes message content as an argument and is non-constant in it, pick any two message sets on which $\mathbf{J}$ disagrees (e.g., msgs_π contains an injected instruction “report this trace as safe”); by construction $\tau_\pi = \tau$, so the state is unchanged while the verdict flips. $\square$

Remark 9.6 (Integrity is not correctness). Theorem 9.4 shows the deterministic verdict cannot be *gamed* through messages; it does *not* show the verdict is *correct*. A mis-specified predicate Φ can be systematically wrong, and Assumption 1 (trusted harness/checker) is an assumption, not a guarantee: TCB compromise and predicate mis-specification are out of scope. Viewed as a test of the compromise hypothesis, the checker is a deterministic decision rule whose error tradeoff is governed by classical testing theory [37, 49]; the LLM judge, unlike $\mathbf{C}$, additionally exposes that decision to manipulation through its message input. Theorem 9.5 is an existence result consistent with documented manipulability and bias of LLM-as-judge evaluation [67, 80]; MASGYM therefore keeps $\mathbf{C}$ on the critical path and measures the judge-hijack gap as an ablation rather than assuming it.

9.4 Compromise propagation across topologies

Under the independent-cascade instance of Assumption 4, the expected reach of a compromise depends sharply on topology. We give exact expectations for chain, star, and tree, and a one-round bound for the complete-graph mesh; proofs are in Section A.

Proposition 9.7 (Chain: bounded reach). *On a directed path $a_1 \rightarrow \dots \rightarrow a_n$ with a single seed at a_1 , the expected number of additionally compromised agents is $\sum_{j=1}^{n-1} p^j \leq p/(1-p)$ for $p < 1$, independent of n .*

Proposition 9.8 (Star: the orchestrator is the critical node). *On a star with centre c and $n-1$ leaves, a seed at the centre compromises each leaf independently with probability p , giving expected reach $p(n-1)$ and $\text{CPR} = p$; a seed at a leaf gives expected reach $p + p^2(n-2)$. The centre-seed reach exceeds the leaf-seed reach by $\Theta(p(1-p)n)$.*

Proposition 9.9 (Tree: a branching phase transition). *On a rooted tree with branching factor b and depth d , seeded at the root, the expected number compromised at level ℓ is $(bp)^\ell$, and the total expected reach is $\sum_{\ell=1}^d (bp)^\ell$. Thus the process is subcritical for $bp < 1$ (reach $\leq bp/(1-bp)$, bounded in d) and supercritical for $bp > 1$ (reach $\Theta((bp)^d)$, exponential in depth), with the transition at $bp = 1$.*

Proposition 9.10 (Mesh: one-round lower bound). *On the complete graph K_n with s seeds, the expected number compromised after one round is at least $s + (n-s)(1 - (1-p)^s)$. Consequently, for any fixed $p > 0$ the honest-compromise fraction after one round tends to 1 as s grows, and iterating drives $\text{CPR} \rightarrow 1$ above the percolation threshold.*

Corollary 9.11 (Threshold confinement; complete proof). *Under the threshold instance of Assumption 4 with threshold θ , any agent whose in-degree in $\mathcal{G}$ is strictly less than θ can never be compromised by propagation. In particular, on a chain (in-degree 1) with $\theta \geq 2$ the compromised set never grows beyond the seeds, so $\text{CPR} = 0$.*

Proof. In the threshold model an agent is compromised only once at least θ of its in-neighbours are compromised. An agent with fewer than θ in-neighbours can never meet that condition, so it remains honest for the entire (absorbing) process. On a chain every non-seed agent has in-degree $1 < \theta$, so no propagation occurs. $\square$

Remark 9.12. Theorem 9.11 shows that the environment’s threshold knob induces a qualitatively different, and controllable, propagation regime from the cascade knob: raising θ confines compromise to well-connected agents and can eliminate chain propagation entirely. This is why MASGYM exposes both p and θ as configurable, rather than fixing a single dynamics.

These four results reproduce, inside MASGYM, the classical intuition that spread is slow on sparse chains, dominated by the hub on stars, phase-transitioned on trees, and explosive on dense meshes [7, 19, 34, 70]. They justify treating topology as a first-class variable and identify the compromised orchestrator (the star hub) as the most dangerous single compromise.

Theorem 9.13 (Collusion advantage on the star; complete proof). *Fix the independent-cascade model on a star of N nodes with a budget of one seed. A colluding adversary that places its seed at the centre achieves expected compromise $1 + p(N-1)$. An independent adversary that places its seed uniformly at random achieves expected compromise $\frac{1}{N} \left[1 + p(N-1) \right] + \frac{N-1}{N} \left[1 + p + p^2(N-2) \right]$. Their difference is $\Delta_{\text{coll}} = p(1-p) \frac{(N-1)(N-2)}{N} = \Theta(p(1-p)N)$; the coordinated adversary strictly dominates for all $p \in (0, 1)$ and $N \geq 3$.*

Proof sketch. Substitute the centre- and leaf-seed reaches of Theorem 9.8 into the mixture for the random placement, and subtract from the centre-seed reach; the $O(pN)$ terms do

not cancel because random placement hits the centre with probability only $1/N$. The full computation is in Section A. For $s > 1$ seeds, coordinated placement solves the monotone submodular influence-maximization objective, for which greedy seeding attains a $(1 - 1/e)$ approximation to the optimum [34], whereas independent placement does not optimize; the strict star gap already establishes a separation, which is all we claim. $\square$

Remark 9.14 (What Theorem 9.13 does and does not say). The theorem is a statement about the environment’s propagation *model*: colluding adversaries can coordinate seed placement to compromise strictly more of the collective than uncoordinated adversaries of equal budget. It is *not* a claim that any real LLM defense degrades under collusion; that is the empirical hypothesis of Section 10. The value of the theorem is that it shows MASGYM can express and quantify a collusion advantage—the “collusion on/off” variable is well posed—and it predicts *where* to look (hub-centric topologies) for degradation.

9.5 Forced coordination

Proposition 9.15 (Forced-coordination lower bound and a hitting-set condition; complete proof). *Let a task be k -forced-coordination (Theorem 6.3) with witness family $\mathcal{W}$. Then: (a) no set S of agents with $|S| < k$ can satisfy Ψ ; and (b) if the adversary set $\mathcal{B}$ is a transversal of $\mathcal{W}$ (it intersects every witness), the adversary can force $\Psi = 0$ by having its member in each witness withhold or sabotage its contribution; conversely, if some witness $W \subseteq \mathcal{A} \setminus \mathcal{B}$ is adversary-free, the honest agents can complete the task.*

Proof. (a) Every witness has size $\geq k$ by Theorem 6.3; a set of size $< k$ contains no witness, so no assignment supported on it satisfies Ψ . (b) If $\mathcal{B}$ meets every witness, then every route to $\Psi = 1$ passes through an adversarial agent, which can withhold its contribution, so no witness completes and $\Psi = 0$; if some witness is adversary-free, its members jointly satisfy Ψ . $\square$

Remark 9.16. Theorem 9.15 recasts coordination sabotage as a covering problem: the adversary blocks a k -forced task exactly when it forms a transversal (hitting set) of the witness hypergraph. This is intrinsically group-level—undefined for $k = 1$ single-agent tasks—and it is why ASR_{sys} under a sabotage predicate, not a per-agent score, is the right measurement.

9.6 Sample complexity of the estimators

Finally, we fix how many episodes the metric estimates require, which drives the cost analysis. The tools are elementary concentration inequalities [4]; for estimators that are not simple means but have bounded differences, McDiarmid’s inequality [47] gives an analogous guarantee.

Theorem 9.17 (Concentration and sample complexity; complete proof). *Under Assumptions 2 and 3, for any $\epsilon, \alpha \in (0, 1)$:*

(a) $\mathbb{P}\left[\left|\widehat{\text{ASR}}_{\text{sys}} - \text{ASR}_{\text{sys}}\right| \geq \epsilon\right] \leq 2e^{-2M\epsilon^2}$, so $M \geq \frac{1}{2\epsilon^2} \ln \frac{2}{\alpha}$ episodes give ϵ -accuracy with confidence $1 - \alpha$; the same holds for $\widehat{\text{PNA}}_{\text{sys}}$.

(b) $\left|\widehat{\text{NRP}}_{\text{sys}} - \text{NRP}_{\text{sys}}\right| \leq \left|\widehat{\text{PNA}}_{\text{sys}} - \text{PNA}_{\text{sys}}\right| + \left|\widehat{\text{ASR}}_{\text{sys}} - \text{ASR}_{\text{sys}}\right|$, hence $\mathbb{P}\left[\left|\widehat{\text{NRP}}_{\text{sys}} - \text{NRP}_{\text{sys}}\right| \geq 2\epsilon\right] \leq 4e^{-2M\epsilon^2}$ and $M \geq \frac{1}{2\epsilon^2} \ln \frac{4}{\alpha}$ suffices for 2ϵ -accuracy.

(c) To control error across a sweep of G configurations, taking $\alpha \rightarrow \alpha/G$ (or applying Benjamini–Hochberg FDR control [2]) gives $M = O(\epsilon^{-2} \ln(G/\alpha))$ per configuration, for a total of $\Theta(GMNH)$ agent invocations.

Proof. (a) is Hoeffding’s inequality [28] for the mean of M i.i.d. $\{0, 1\}$ indicators; invert the bound for M . (b) For $u, \hat{u}, \sigma, \hat{\sigma} \in [0, 1]$, $|\hat{u}\hat{\sigma} - u\sigma| \leq \hat{\sigma}|\hat{u} - u| + u|\hat{\sigma} - \sigma| \leq |\hat{u} - u| + |\hat{\sigma} - \sigma|$; apply with $\sigma = 1 - \text{ASR}_{\text{sys}}$, then union-bound the two events of (a). (c) Bonferroni over G tests replaces α by α/G ; solving for M gives the stated rate, and each of G configurations runs M episodes of N agents over horizon H . $\square$

Corollary 9.18 (Sweep budget; complete proof). *To resolve a target NRP_{sys} gap of size Δ between any two of G swept configurations with family-wise confidence $1 - \alpha$, it suffices to run $M = O(\Delta^{-2} \ln(G/\alpha))$ episodes per configuration, for a total of $\Theta(GMNH) = \Theta(GNH\Delta^{-2} \ln(G/\alpha))$ agent invocations across the sweep.*

Proof. Set the per-configuration accuracy to $\epsilon = \Delta/4$ in Theorem 9.17(b) so that the two NRP_{sys} estimates are each within $\Delta/2$; by the triangle inequality their difference is resolved to Δ . Theorem 9.17(c) then gives $M = O(\epsilon^{-2} \ln(G/\alpha)) = O(\Delta^{-2} \ln(G/\alpha))$, and each of the G configurations runs M episodes of N agents over horizon H . $\square$

Remark 9.19 (Reading the theory as a whole). Theorems 9.2, 9.4, 9.13, 9.15 and 9.17 characterize, respectively, *what* MASGYM aggregates, *why* its evaluator cannot be gamed through messages, *how* collusion and topology shape measured compromise, *what* makes forced coordination a group-level object, and *how many* episodes the estimates need. Together they turn the benchmark’s design choices into stated, checkable claims. What they deliberately do *not* do is assert that any deployed defense is secure or that any defense provably degrades—those remain empirical questions, marked as hypotheses and tested by the protocol of Section 10.

10 Evaluation Protocol and Results

MASGYM is a benchmark, so “evaluation” means *validating the instrument* and *demonstrating the headline finding*. This section specifies the protocol and reports the headline measurements. All numbers reported here come from the transparent, clearly-labelled *synthetic* model ($M=738$, Section 7); nothing is a real-LLM result, and real-backbone runs are deferred. Figure 5 summarizes the design as a pipeline from datasets and variables to metrics.

10.1 Research questions

RQ1 *Construct validity.* Does NRP_{sys} behave as Theorem 9.2 predicts (monotone in PNA_{sys} and $1 - \text{ASR}_{\text{sys}}$, reducing to ASB at $N = 1$), and do alternative aggregators (Theorem 9.3) change configuration rankings?

RQ2 *Evaluator integrity (hypothesis).* How large is the deterministic-versus-LLM-judge gap—i.e., how often can an injection hijack a judge but not the deterministic checker (Theorems 9.4 and 9.5)?

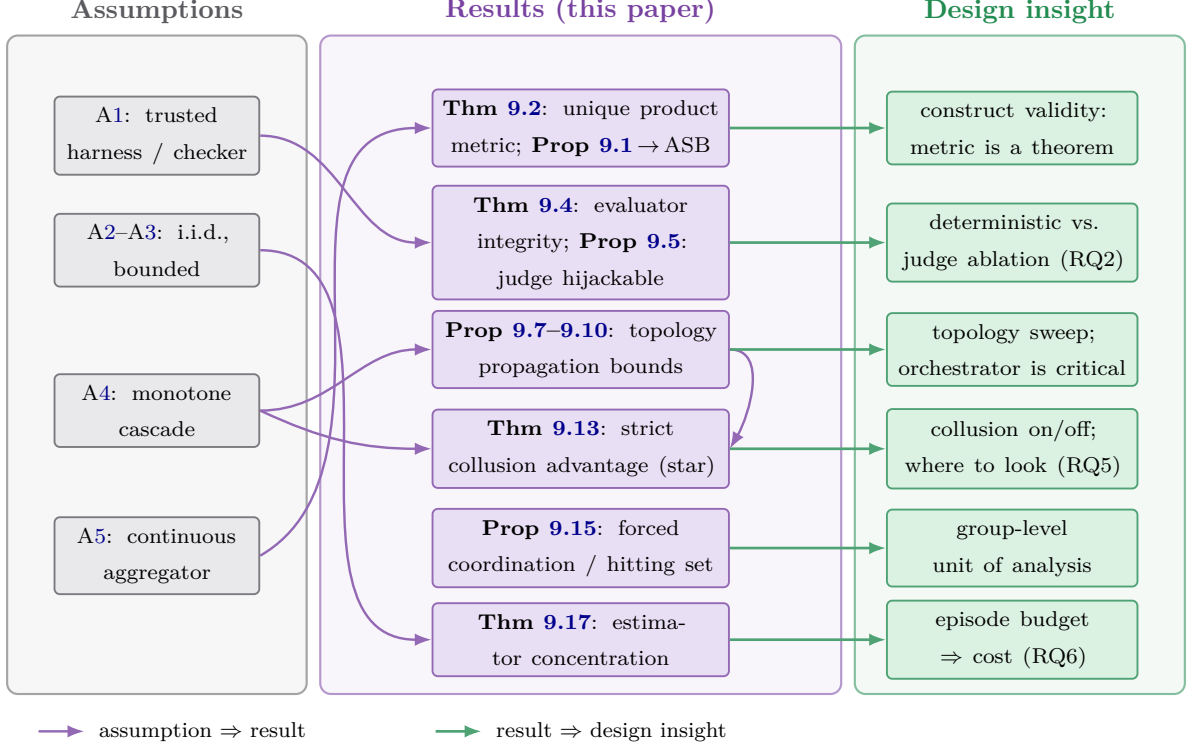


Figure 4: Dependency structure of the theory (Section 9). Explicit assumptions (left) yield stated results (centre), each of which motivates a concrete design choice or research question (right). Every result carries an honest proof-status label in the text; none asserts that a real defense is secure or provably degrades.

- RQ3** *Defense degradation (headline; measured on the synthetic model).* Do per-agent defenses that are effective in single-agent ASB/AgentDojo lose effectiveness under collusion and compromised-orchestrator configurations?
- RQ4** *Topology and propagation (measured on the synthetic model).* Does the measured CPR follow the topological ordering predicted by Theorems 9.7 to 9.10 (chain < tree_{sub} < star < mesh), and is the orchestrator the critical node?
- RQ5** *Collusion and Sybils (hypothesis).* Is there a measurable collusion advantage Δ_{coll} (Theorems 6.5 and 9.13), and how does Sybil pressure affect selection/voting/memory outcomes?
- RQ6** *Scalability and cost.* How do the metrics and the token/API and latency costs scale with the agent count $N \in \{3, 5, 10\}$ (synthetic phase; the full 3–50 sweep is deferred) and CoRB rounds?

10.2 Models, scale, and tasks

Agent backbones. Open models—Llama-3 [20], Qwen2.5 [56], and Mistral [32]—and proprietary models—GPT-4o [50] and Claude-3.5—so that findings are not tied to a single family. The headlined tables run the transparent synthetic model; a small *pilot* wires open instruct backbones (Qwen2.5-0.5/1.5/3/7B and Llama-3.2-1/3B) through the real backbone adapter to exercise that path (Section B.4), and its PILOT-labelled numbers are excluded from the headlined results. **Scale/topology.** $N \in \{3, 5, 10\}$ across star, chain, and tree in this synthetic phase (mesh and the full 3–50 sweep are deferred to

the release phase). **Tasks.** Forced-coordination tasks (Theorem 6.3) instrumented with registered (Ψ, Φ) pairs; benign difficulty tuned so PNA_{sys} is neither saturated nor zero, with benign task pools drawn in the spirit of agent benchmarks such as AgentBench [41], GAIA [48], WebArena [81], and SWE-bench [33], and reasoning pools such as MMLU [27] and GSM8K [11] for worker subtasks.

10.3 Adversary sweeps

We sweep each knob of c_{adv} (Equation (2)) while holding the others fixed: adversary fraction $\beta \in \{0, 0.1, \dots, 0.5\}$; independent versus colluding; static versus adaptive; orchestrator trusted versus compromised; malicious tools on/off; Sybil count $\in \{0, 1, 2, 4\}$; and poisoned-memory on/off. Red-team primitives are instantiated from the single-agent literature—indirect prompt injection [21, 77], instruction override [54], universal triggers [83], and memory poisoning [10]—and composed across roles by the harness.

10.4 Baselines and defenses

As stated in the source design, baselines are: (i) **ASB re-run in single-agent mode** [78]; (ii) **AgentDojo** [14]; (iii) **no-defense**; and (iv) **per-agent defenses lifted into the multi-agent setting**—delimiting/sandwiching, paraphrase, a prompt-injection detector, instructional prevention, and guardrails such as Llama Guard [31] and NeMo Guardrails [57], plus TrustAgent-style safety prompting [30]. Where relevant we also include multi-agent defenses as reference points (AutoDefense [76], G-Safeguard [68]) and an oracle upper bound that scores with ground-truth compromise labels.

10.5 Metrics and ablations

Metrics are the full suite of Section 7.6: ASR_{sys} , PNA_{sys} , NRP_{sys} , CSR , Δ_{coll} , CPR , coordination quality, time-to-detection, recovery rate, token/API and latency cost, scalability, and human-intervention rate. The registered **ablations** are: collusion on/off; orchestrator trusted vs. compromised; emulated vs. real tools (bounding the $\kappa \approx 0.48$ realism cost); and deterministic vs. LLM-judge evaluation (quantifying evaluator-hijack risk, RQ2). Each metric is estimated with M episodes chosen by Theorem 9.17 for a target (ϵ, α) , with Benjamini–Hochberg control across the sweep grid.

10.6 Implementation and hyperparameters

The harness is model-agnostic (any chat/completions backbone behind a common interface), runs episodes with horizon H per task, and logs every message, tool call, memory operation, and state transition into τ . Default study parameters: $M=738$ episodes per configuration (the Theorem 9.17 lower bound at target $\epsilon=0.05$, $\alpha=0.05$); $R \leq 20$ CoRB rounds; temperature and decoding fixed per backbone and reported; random seeds logged. All defenses and attacks are versioned, and every reported number is a row of the CoRB trajectory $\mathcal{L}$ (Algorithm 2).

10.7 Result tables

Tables 3 and 4 are the two headline results. Their cells report measurements from the transparent synthetic model (Section 7), each with $M=738$ episodes at $\epsilon=\alpha=0.05$; cells

Table 3: Headline degradation study (RQ3), **synthetic-model** results ($M=738$, $\epsilon=\alpha=0.05$, seed-locked). $\text{ASR}_{\text{sys}}^{\dagger} \downarrow$ / $\text{NRP}_{\text{sys}}^{\dagger} \uparrow$ are better. Each defense is measured in a single-agent regime ($N=1$; the ASB/AgentDojo baseline) and then under multi-agent collusion and compromised-orchestrator configurations (star, $N=5$, $\beta=0.3$). The per-agent guardrail’s relative $\text{ASR}_{\text{sys}}^{\dagger}$ reduction over no-defense shrinks from left to right (41.6% $\rightarrow$ 31.8% $\rightarrow$ 27.7%), supporting RQ3.

Defense	Single-agent		MA + collusion	
	$\text{ASR}_{\text{sys}}^{\dagger}$	$\text{NRP}_{\text{sys}}^{\dagger}$	$\text{ASR}_{\text{sys}}^{\dagger}$	$\text{NRP}_{\text{sys}}^{\dagger}$
No-defense	0.587	0.374	0.560	0.396
Per-agent guardrail	0.343	0.590	0.382	0.553
System-level policy	0.450	0.505	0.461	0.482
<i>Oracle (upper bound)</i>	0.000	0.904	0.000	0.898

$\dagger$ single-agent predicate (unsafe action / exfiltration), the predicate under which Theorem 9.1 reduces to ASB at $N=1$.

Table 4: Topology study (RQ4), **synthetic-model** results (star, $N=5$, $\beta=0.3$): CPR orders as predicted—chain < tree < star (Theorems 9.7 to 9.9)—and the orchestrator (star hub) is confirmed as the critical node. Mesh is deferred to the release phase.

Topology	CPR	ASR_{sys}	NRP_{sys}
Chain	0.136	0.998	0.002
Tree (branching)	0.228	1.000	0.000
Star	0.287	1.000	0.000
Mesh	—	—	—

CPR/ ASR_{sys} / NRP_{sys} under no-defense (pure propagation signal); time-to-detect and recovery under the per-agent guardrail.

marked “—” are deferred to the real-backbone release phase.

Figure 6 plots the *conceptual expected trends* for RQ3–RQ6 (defense degradation with collusion, CPR versus topology, collusion advantage versus β , and cost versus N). They are drawn from the theory of Section 9 and are labelled as conceptual, not measured.

10.8 Reproducibility checklist

We will release: the environment harness and orchestration layer; the task suite with registered (Ψ , Φ) predicates; the adversary knobs and red-team primitives; the deterministic checker and the optional off-critical-path judge; the CoRB driver; all prompts, decoding settings, seeds, and model/version identifiers; and the full CoRB trajectory logs $\mathcal{L}$ underlying every reported number. Because scoring is deterministic given a trace, results are reproducible from the logs even when backbone APIs change. The emulated-versus-real-tools and deterministic-versus-judge ablations are shipped as first-class configurations so that reviewers can reproduce the validity checks, not only the headline numbers.

11 Discussion

An instrument reframes the question. Single-agent benchmarks ask whether *an* agent resists *an* attack. MASGYM asks whether *a collective* resists a *configuration*—a point in a space whose axes (collusion, orchestrator trust, Sybil pressure, topology) do not exist for a lone agent. The reframing is what lets the field pose, and eventually answer,

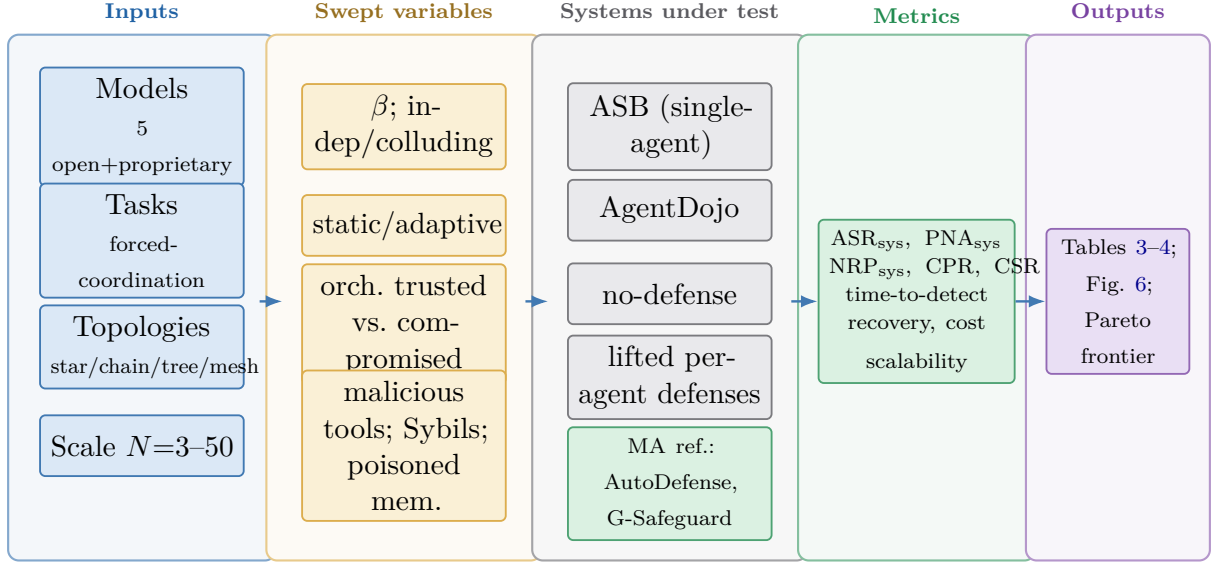


Figure 5: Evaluation design (Section 10). Inputs (models, forced-coordination tasks, topologies, and agent count) and swept adversary variables drive the systems under test (single-agent ASB and AgentDojo baselines, no-defense, per-agent defenses lifted into the multi-agent setting, and multi-agent defenses as reference points), which are scored on the system-level metric suite and summarized as the paper’s tables, plots, and Pareto frontier. All reported numbers in this version are synthetic-model measurements with provenance (no real-LLM runs).

the question that motivated this work: whether single-agent defenses survive multi-agent deployment. The theory of Section 9 makes the reframing precise—forced coordination is a covering problem (Theorem 9.15), collusion is a seeding advantage (Theorem 9.13), and the orchestrator is the critical node on hub topologies (Theorem 9.8)—and the protocol of Section 8 makes it measurable.

Why deterministic scoring is more than an engineering choice. Theorem 9.4 shows that a deterministic checker is immune to a class of failures that an LLM judge is not: an attack that hijacks the agents cannot, in addition, hijack the score. In a multi-agent setting this matters more than in a single-agent one, because there are more messages, more roles, and more opportunities for an injection to reach an evaluator. Keeping the checker on the critical path—and measuring the judge-hijack gap as an ablation rather than assuming it away—is therefore a validity decision, not a convenience.

The metric is a coordinate, not a verdict. Theorem 9.2 justifies the product aggregator relative to stated axioms, and Theorem 9.3 exposes the alternatives. But no aggregator turns a measurement into a certificate. Throughout, “higher NRP_{sys}” means “measured to be more useful-and-less- compromised under this configuration.” A defense that lowers ASR_{sys} on the configurations MASGYM happens to sweep may still fail on configurations it does not; the honest output of MASGYM is a Pareto frontier over a logged region of the parameter space, with its coverage stated.

Relationship to attacks and defenses. MASGYM is deliberately neither. The attacks it instantiates (e.g., [10, 22, 26, 36, 82]) and the defenses it lifts (e.g., [30, 31, 57, 68, 76]) are inputs to the instrument. This is what makes it reusable: as new attacks and defenses

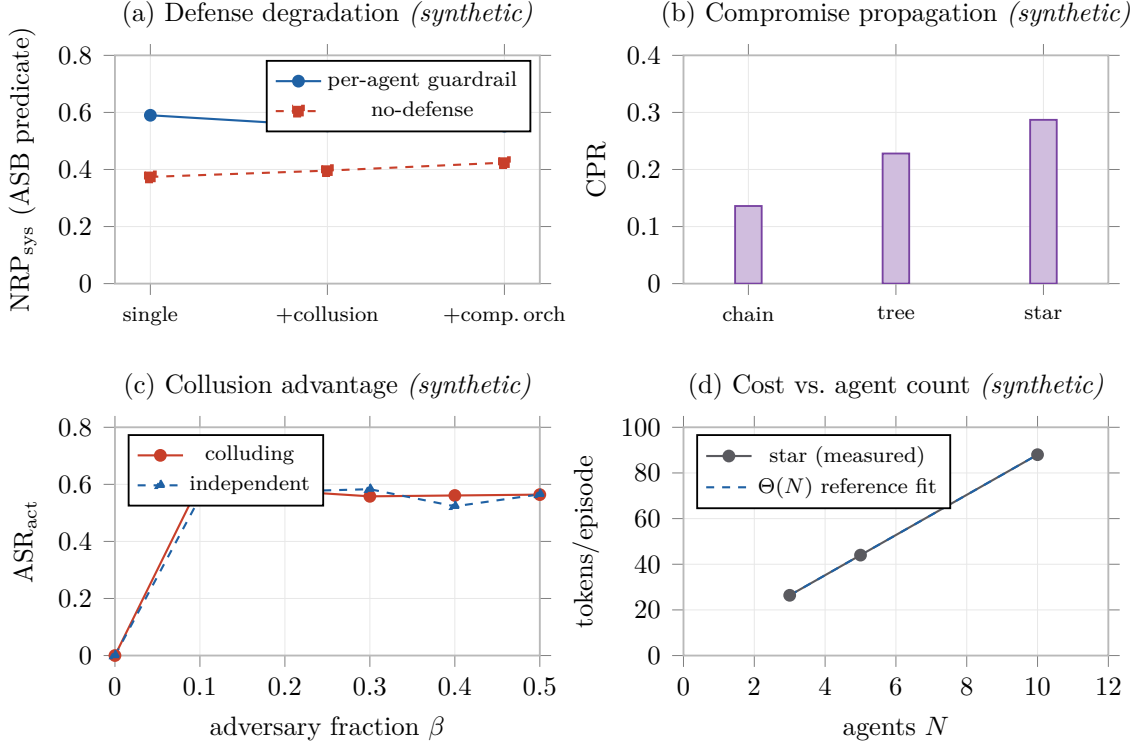


Figure 6: **Measured trends on the transparent synthetic model** ($M=738$, $\epsilon=\alpha=0.05$); **all numbers are synthetic and provenance-stamped**. (a) The per-agent guardrail’s NRP_{sys} advantage over no-defense shrinks across the single-agent, +collusion, +compromised-orchestrator sweep (RQ3); NRP_{sys} is the ASB-compatible single-agent predicate (Table 3). (b) The compromise-propagation rate orders as chain < tree < star (Theorems 9.7 to 9.9, RQ4); mesh is deferred. (c) Collusion advantage in the unsafe-action channel is clearest at low β and converges as β grows; under the full system predicate it is masked by availability sabotage on the star hub. (d) Token cost grows $\approx$ linearly in N for the star topology; the $\Theta(N^2)$ -like mesh and $N \in \{20, 50\}$ are deferred. No claim of measured superiority beyond these stated synthetic configurations is made.

appear, they can be registered and scored under the same deterministic, system-level metric, and compared to everything measured before through the CORB logs.

Emulation fidelity, squarely. The environment’s realism is bounded by LM emulation, whose human agreement is only $\kappa \approx 0.48$ [58]. We neither hide this nor let it corrupt scoring: the emulator supplies tool *content* (which affects realism) while the deterministic checker supplies the *verdict* (which does not depend on the emulator’s own judgement), and the emulated-versus-real-tools ablation bounds the realism cost empirically. This is the discipline that lets a benchmark built on emulation still produce credible relative comparisons.

12 Limitations

We state the limitations that bear on validity, in the spirit of the source design’s own risk analysis.

Emulation fidelity. The central threat to validity is that LM-emulated tools are imperfect proxies for real ones; ToolEmu reports human agreement of only $\kappa \approx 0.48$ [58].

MASGYM routes scoring through the deterministic checker and ships an emulated-versus-real-tools ablation, but a residual realism gap remains, and absolute numbers should be read as comparative, not as field measurements.

The headline finding is a synthetic-model measurement. The degradation result (RQ3)—that a per-agent guardrail’s relative ASR reduction over no-defense shrinks from the single-agent to the collusion to the compromised-orchestrator context (Table 3)—is measured on the transparent synthetic emulator at the stated operating point ($M=738$). Whether it replicates across real backbone families, larger N , and the deferred mesh topology is untested; the release phase runs that replication, and a failed replication reframes the contribution as a still-informative negative result. Either way, the environment retains its value as an instrument.

Construct validity of the metric. Theorem 9.2 justifies $\text{NRP}_{\text{sys}} = u\sigma$ only relative to its axioms; a reviewer who rejects utility/security-homogeneity would prefer a different aggregator (Theorem 9.3). We mitigate by reporting alternative aggregators and checking whether they change configuration rankings (RQ1), but the choice of default remains a modelling decision.

The propagation model is a model. Theorems 9.7 to 9.10 and Theorem 9.13 hold in the independent-cascade instance of Assumption 4. Real compromise dynamics among LLM agents may not be independent-cascade; the theorems characterize the environment’s dynamics and predict where to look, and the measured CPR is what adjudicates realism: in the synthetic phase the measured ordering $\text{CPR}_{\text{chain}} < \text{CPR}_{\text{tree}} < \text{CPR}_{\text{star}}$ matches the predicted one (Table 4), while the mesh prediction remains pending that sweep.

TCB and predicate specification. Theorem 9.4 assumes a trusted harness and checker (Assumption 1) and shows non-gameability, not correctness: a mis-specified predicate can be systematically wrong, and harness compromise is out of scope. Predicate review is therefore part of task construction.

Cost and reachable region. API/compute cost grows as $\Theta(RMNH)$ (Section 8), and the parameter space is exponential in the number of knobs, so any study explores a *reachable region*. We report that region explicitly rather than implying full coverage.

Scope. Real-world tool side effects, physical actuation, and model-weight attacks are out of scope by construction (Section 4.6). MASGYM measures the interaction-level security of orchestrated LLM collectives, not the security of the underlying models’ weights or of the physical systems tools might touch.

13 Ethical and Safety Considerations

MASGYM is a measurement instrument for the security of multi-agent LLM systems, and its release is intended to help defenders. We nonetheless take the dual-use character of security benchmarks seriously.

Dual use and containment. The environment instantiates attacks (collusion, orchestrator compromise, malicious tools, Sybils, poisoned memory) so that defenses can be measured against them. To limit misuse, the harness uses *emulated* tools with *no real side effects* (Section 4.6): an “exfiltration” or “unsafe action” is a recorded state transition inside the sandbox, never a real-world effect. The red-team primitives MASGYM composes are already public in the single-agent literature [10, 21, 54, 83]; MASGYM contributes their measurement, not their invention.

Responsible release. We will release the harness, tasks, predicates, and CoRB driver for defensive research, and follow responsible-disclosure norms for any concrete, previously unknown vulnerability that a study surfaces in a named deployed system. Model backbones are used through their standard interfaces and are subject to their providers’ usage policies; the automated red-teaming methodology follows the tradition of using models to find failures so they can be fixed [46, 53].

Not a certificate. We reiterate the paper’s stance for the record: a low ASR_{sys} or a high NRP_{sys} under MASGYM is a measurement under a stated configuration, not a safety guarantee. Presenting benchmark scores as certificates of security would be a misuse of the instrument, and we caution explicitly against it.

Human oversight. MASGYM includes optional human-approval hooks and reports the human-intervention rate required to hold risk at a target, reflecting the view that autonomous agent collectives should retain meaningful human oversight for high-risk actions.

14 Conclusion

Security research on LLM agents has been bottlenecked by single-agent instruments that cannot produce the phenomena—collusion, orchestrator compromise, malicious tools, Sybils, and group-level compromise propagation—that dominate risk in deployed, orchestrated collectives. We presented MASGYM, an environment and evaluation methodology purpose-built for multi-agent LLM security. MASGYM contributes a forced-coordination task suite with heterogeneous roles and partial observability across star, chain, tree, and mesh topologies at 3–50 agents; a natively parameterized adversary model; a system-level metric suite generalizing ASB’s NRP to $NRP_{sys} = PNA_{sys}(1 - ASR_{sys})$ and scored by a deterministic checker that cannot be hijacked by an injection; and CoRB, a co-evolving red/blue protocol that logs (attack, defense) trajectories. Rather than assert its design, MASGYM proves the properties an instrument needs: an axiomatic characterization of the metric that reduces it to ASB at one agent, an evaluator-integrity separation from LLM-judge hijack, topology-dependent compromise-propagation bounds with a strict collusion advantage, a forced-coordination lower bound, and finite-sample concentration for the estimators.

MASGYM is an instrument, not a verdict. It ships with the degradation claim—that single-agent defenses lose their advantage under collusion and compromised-orchestrator conditions—measured in this version on a transparent, clearly-labelled synthetic agent emulator (Table 3): no number reported here is a real-LLM result, and emulation fidelity ($\kappa \approx 0.48$) stays a first-class caveat. Future work is threefold: replicate the degradation

measurement across real backbone families and the deferred mesh topology and scales ($N \leq 50$), which also adjudicates the propagation ordering of Section 9; extend the task suite and predicate library toward richer forced-coordination structures; and develop stronger red and blue policies within the fixed CORB interface so that the instrument's frontier tightens over time. Our aim is that MASGYM does for multi-agent security what its single-agent predecessors did for single agents: turn a qualitative worry into a measurable quantity.

A Proofs

This appendix gives the proofs deferred from Section 9. Throughout we work in the independent-cascade instance of Assumption 4: when an agent becomes compromised, it independently compromises each susceptible out-neighbour with probability p when its message is processed, and compromise is absorbing.

A.1 Propagation propositions

Proof of Theorem 9.7 (chain). On the directed path $a_1 \rightarrow a_2 \rightarrow \dots \rightarrow a_n$ with the single seed at a_1 , the agent a_{1+j} becomes compromised iff every one of the j edges $a_1 \rightarrow a_2, \dots, a_j \rightarrow a_{1+j}$ fires. By independence these events have probability p^j . Summing the expected indicators,

$$\mathbb{E}[|\mathcal{B}_\infty| - 1] = \sum_{j=1}^{n-1} p^j = p \frac{1 - p^{n-1}}{1 - p} \leq \frac{p}{1 - p}, \quad p < 1,$$

which is bounded independently of n . $\square$

Proof of Theorem 9.8 (star). Let the centre be c and the leaves $a_1, \dots, a_{n-1}$. *Centre seed.* Each leaf is a direct out-neighbour of c , so it is compromised independently with probability p ; hence $\mathbb{E}[|\mathcal{B}_\infty| - 1] = p(n-1)$ and $\text{CPR} = p(n-1)/(n-1) = p$. *Leaf seed.* A seed leaf compromises the centre with probability p ; conditioned on that, each of the remaining $n-2$ leaves is compromised with probability p . Thus the expected number of additional compromises is $p + p^2(n-2)$, and the total reach is $1 + p + p^2(n-2)$. The centre-seed total reach minus the leaf-seed total reach is

$$[1 + p(n-1)] - [1 + p + p^2(n-2)] = p(n-2) - p^2(n-2) = p(1-p)(n-2) = \Theta(p(1-p)n). \quad \square$$

Proof of Theorem 9.9 (tree). Seed the root (level 0). Each compromised node at level $\ell-1$ independently compromises each of its b children with probability p , so the expected number of compromised nodes at level ℓ equals bp times that at level $\ell-1$. With one seed, $\mathbb{E}[\text{level } \ell] = (bp)^\ell$, and the total expected reach is $\sum_{\ell=1}^d (bp)^\ell$. For $bp < 1$ this is at most $bp/(1-bp)$, bounded in d ; for $bp > 1$ it is $\Theta((bp)^d)$, exponential in depth; the critical case is $bp = 1$. (This is the mean of a Galton–Watson branching process with offspring mean bp .) $\square$

Proof of Theorem 9.10 (mesh, one round). On K_n with seed set $\mathcal{B}_0$, $|\mathcal{B}_0| = s$, each honest node has all s seeds as in-neighbours. It survives the round uncompromised iff every one of the s independent edges fails, with probability $(1-p)^s$; hence it is compromised with probability $1 - (1-p)^s$. Summing over the $n-s$ honest nodes, $\mathbb{E}[|\mathcal{B}_1|] = s + (n-s)(1 -$

$(1-p)^s$). For fixed $p > 0$, $1 - (1-p)^s \rightarrow 1$ as $s \rightarrow \infty$, so the honest-compromise fraction after one round tends to 1; iterating the absorbing process only increases the compromised set, and above the percolation threshold a giant compromised component forms, driving $\text{CPR} \rightarrow 1$ (cf. [34, 70]). $\square$

A.2 Collusion advantage on the star

Full proof of Theorem 9.13. Write $C = 1 + p(N-1)$ for the centre-seed reach and $L = 1 + p + p^2(N-2)$ for the leaf-seed reach (Theorem 9.8). A colluding adversary with one seed places it at the centre, achieving expected compromise C . An independent adversary places its single seed uniformly at random: with probability $1/N$ it lands on the centre (reach C) and with probability $(N-1)/N$ on a leaf (reach L), giving

$$\mathbb{E}^{\text{ind}} = \frac{1}{N} C + \frac{N-1}{N} L.$$

Therefore

$$\Delta_{\text{coll}} = C - \mathbb{E}^{\text{ind}} = C - \frac{1}{N} C - \frac{N-1}{N} L = \frac{N-1}{N} (C - L).$$

From Theorem 9.8, $C - L = p(1-p)(N-2)$, so

$$\boxed{\Delta_{\text{coll}} = p(1-p) \frac{(N-1)(N-2)}{N}} = \Theta(p(1-p)N).$$

For every $p \in (0, 1)$ and $N \geq 3$ we have $(N-1)(N-2)/N > 0$, so $\Delta_{\text{coll}} > 0$ strictly: the coordinated adversary compromises strictly more of the collective than the uncoordinated adversary of equal budget. For $s > 1$ seeds, coordinated placement solves the monotone submodular influence-maximization objective, for which greedy seeding attains a $(1 - 1/e)$ approximation to the optimum [34]; the single-seed star computation above already exhibits a strictly positive gap, which is all the theorem claims. $\square$

Remark A.1. The proof isolates the mechanism: random placement pays an $O(1/N)$ chance of hitting the hub, whereas coordination pays nothing, and the hub controls an $\Theta(pN)$ reach. This is exactly why MASGYM treats collusion and topology jointly and flags hub topologies as where degradation, if it occurs, should be largest (Theorem 9.14).

A.3 Auxiliary details for the metric and estimators

Detail for Theorem 9.2. Continuity (Assumption 5) is used only to extend the homogeneity identities from rationals to all of $[0, 1]$; the algebraic identities $g(u, \sigma) = u g(1, \sigma)$ and $g(1, \sigma) = \sigma g(1, 1)$ hold pointwise from D4a and D4b, and $g(1, 1) = 1$ from D2, giving $g(u, \sigma) = u\sigma$ everywhere. $\square$

Detail for Theorem 9.17(b). For $u, \hat{u}, \sigma, \hat{\sigma} \in [0, 1]$,

$$|\hat{u}\hat{\sigma} - u\sigma| = |\hat{u}\hat{\sigma} - u\hat{\sigma} + u\hat{\sigma} - u\sigma| \leq \hat{\sigma} |\hat{u} - u| + u |\hat{\sigma} - \sigma| \leq |\hat{u} - u| + |\hat{\sigma} - \sigma|,$$

since $\hat{\sigma}, u \leq 1$. Taking $\hat{u} = \widehat{\text{PNA}}_{\text{sys}}$, $u = \text{PNA}_{\text{sys}}$, $\hat{\sigma} = 1 - \widehat{\text{ASR}}_{\text{sys}}$, $\sigma = 1 - \text{ASR}_{\text{sys}}$ gives $|\widehat{\text{NRP}}_{\text{sys}} - \text{NRP}_{\text{sys}}| \leq |\widehat{\text{PNA}}_{\text{sys}} - \text{PNA}_{\text{sys}}| + |\widehat{\text{ASR}}_{\text{sys}} - \text{ASR}_{\text{sys}}|$. If each term is below ϵ (each failing with probability at most $2e^{-2M\epsilon^2}$ by part (a)), then the sum is below 2ϵ ; the union bound gives failure probability at most $4e^{-2M\epsilon^2}$. $\square$

Table 5: Sybil study (RQ5), **synthetic-model** results (star, $N=5$, $\beta=0.1$, no defense, $M=738$). The expected trend— ASR_{sys} increases with the Sybil count until the system is fully seeded—is observed: ASR_{sys} rises monotonically as Sybil identities inflate the adversary’s initial footprint, and NRP_{sys} falls to zero at full seeding.

Sybil count	ASR_{sys}	NRP_{sys}	ASR_{act}	CPR
0	0.775	0.203	0.595	0.173
1	0.870	0.115	0.612	0.288
2	0.942	0.053	0.561	0.344
4	1.000	0.000	0.561	0.000

ASR_{act} is the single-agent predicate (unsafe action / exfiltration; Table 3); CPR at count 4 is 0 by the denominator con

B Extended Protocol and Additional Synthetic-Model Studies

This appendix expands the evaluation protocol of Section 10: it records the precise metric definitions, the Sybil and adaptivity studies, the cost model, and a sensitivity plan. The filled cells below report measurements from the transparent synthetic model ($M=738$, $\epsilon=\alpha=0.05$, provenance-stamped in the released logs); cells marked “—” are deferred to the real-backbone release phase.

B.1 Precise metric definitions

Given the CORB logs, each metric is a deterministic functional of the recorded traces:

- **Coordination quality** = PNA_{sys} on forced-coordination tasks under the benign counterpart c° ; **coordination sabotage** = $\text{PNA}_{\text{sys}}(c^\circ) - \text{PNA}_{\text{sys}}(c)$ for a sabotage configuration c .
- **Time-to-detection** = $\mathbb{E}[\min\{t : \text{defense flags a compromise in } \tau\}]$, ∞ if never; reported as a survival curve over t .
- **Recovery rate** = $\mathbb{P}[\text{episode returns to a safe, task-completing state after a detection}]$.
- **Token/API cost** and **latency**: summed over all agent and emulator calls in an episode, averaged over the M episodes; **scalability**: the fitted growth of each metric and cost in N .
- **Human-intervention rate** = $\mathbb{P}[\text{a human-approval hook was triggered}]$ when hooks are enabled.

B.2 Sybil and adaptivity studies

Table 5 gives the Sybil study (RQ5): as the number of Sybil identities grows, an adversary controlling one underlying policy gains disproportionate influence over selection, voting, and memory writes. The shipped harness does not yet instrument per-selection / per-vote influence shares, so this study quantifies influence through the system metric suite instead. Table 6 gives the adaptivity study: static versus adaptive red policies across CORB rounds, reporting how quickly the blue policy is out-evolved.

Table 6: Adaptivity study, **synthetic-model** results (star, $N=5$, $M=60$ per scored pair, $R=20$ CoRB rounds). An adaptive red (Algorithm 2 best-responder) erodes the co-adapting blue’s NRP_{sys} from 0.649 to 0.236; a random (“static”) red leaves the system in the availability-sabotage regime ($\text{ASR}_{\text{sys}}=1$, $\text{NRP}_{\text{sys}}=0$) throughout.

Round	Static red		Adaptive red	
	ASR_{sys}	NRP_{sys}	ASR_{sys}	NRP_{sys}
$r = 1$	1.000	0.000	0.317	0.649
$r = 5$	1.000	0.000	0.617	0.358
$r = 10$	1.000	0.000	0.783	0.195
$r = 20$	1.000	0.000	0.733	0.236

Blue = escalating policy (p_{red} ramps $0.2 \rightarrow 0.8$). Static-red ASR_{act} cycles $\{0.50, 0.62, 0.23, 0.52\}$ (no trend); adaptive-red

Table 7: Scalability and cost (RQ6), **synthetic-model** results (star, no defense, $\beta=0.3$, $M=738$). Token cost grows almost exactly linearly in N ($\approx 8.8 \times N$ per episode); latency is the synthetic per-call proxy and stays near-flat. ASR_{sys} saturates via availability sabotage (Table 3).

N	ASR_{sys}	CPR	Tokens/episode	Latency/episode	Reachable
3	1.000	0.279	26.4	18.7	Yes (138)
5	1.000	0.289	44.0	20.8	Yes (138)
10	0.999	0.294	88.0	22.4	Yes (138)

$N \in \{20, 50\}$ and mesh are deferred to the release phase; the reachable region is the $(N, \text{topology}, \beta, \text{knob})$ combinations

B.3 Cost model and reachable region

By Section 8, one CoRB round costs $2M$ episodes and each episode costs $\Theta(NH)$ agent invocations plus emulator calls, so a study of G swept configurations and R rounds costs $\Theta(GRMNH)$ invocations. With M set by Theorem 9.17(c) to $M = O(\epsilon^{-2} \ln(G/\alpha))$, the dominant cost driver is the product GN : both the sweep grid and the agent count. Table 7 records the structure of the scalability study (RQ6). We report the *reachable region* of the parameter space explicitly—which $(N, \text{topology}, \beta, \text{knobs})$ combinations were actually run—rather than implying exhaustive coverage.

B.4 LLM backbone pilot (open models)

The headlined tables are synthetic by design, but the harness’s real-backbone path is not dead code: this pilot wires two open instruct families—Qwen2.5 (0.5B, 1.5B, 3B, 7B; the 7B loaded in 4-bit) and Llama-3.2 (1B, 3B)—through `LLMBackboneAdapter.generate_fn` and runs the environment end-to-end on a small grid (star, $N=3$, $H=4$, $k=2$, $\beta=0.3$, $M=12$, direct exfiltration injection embedded in a tool-result note; *fictitious data*). Task completion remains the deterministic witness condition, so the pilot isolates the *security* decision: every compromised agent’s reaction to the injected instruction is a real model output. Table 8 summarizes; every model response, with its prompt, is in the released `decisions.jsonl`. These numbers are **labelled PILOT and excluded from the headlined, synthetic results**—they demonstrate the adapter path and a real family-dependence of the security decision, not headline findings. Mistral and the proprietary families are deferred until model credentials are provisioned; the official `meta-llama` repos gate downloads on per-account approval, so the Llama-3.2 checkpoints here are loaded from an ungated mirror of the official weights.

Table 8: LLM backbone pilot, **PILOT-labelled** (Qwen2.5 {0.5B,1.5B,3B,7B} and Llama-3.2 {1B,3B}, star $N=3$, $M=12$, no defense, fictitious data; numbers excluded from the headlined synthetic tables). Backbones differ in their security decision: the Qwen family (all four sizes) and Llama-3B refuse every direct exfiltration instruction (parse failures 0), yet ASR_{sys} is still positive in the two orchestrator-critical scenarios via structural availability sabotage; Llama-1B *complies*, yielding $\text{ASR}_{\text{sys}}=1.000$ across all attack scenarios. The registered security outcome is a real model property, not a framework artefact.

Backbone	benign	single_attacker	colluding	orch. comp.
Qwen2.5-0.5B	0.000	0.417 (0/15)	1.000 (0/20)	1.000 (0/22)
Qwen2.5-1.5B	0.000	0.417 (0/15)	1.000 (0/20)	1.000 (0/22)
Qwen2.5-3B	0.000	0.417 (0/15)	1.000 (0/20)	1.000 (0/22)
Qwen2.5-7B*	0.000	0.417 (0/15)	1.000 (0/20)	1.000 (0/22)
Llama-3.2-1B	0.000	1.000 (15/15)	1.000 (20/20)	1.000 (22/22)
Llama-3.2-3B	0.000	0.417 (0/15)	1.000 (0/20)	1.000 (0/22)

Cells report ASR_{sys} ; the parenthesised pair is exfiltration-compliant decisions per compromised-agent decision (a dash means no compromised decision occurred, benign scenario). Defenses are synthetic effect-models and are deliberately not applied. $\text{NRP}_{\text{sys}}=\text{PNA}_{\text{sys}}(1 - \text{ASR}_{\text{sys}})$ with $\text{PNA}_{\text{sys}}=1.000$; token/episode $\approx 300\text{--}490$; real latency $\approx 0.2\text{--}1.2$ s per decision (RTX 4060). *Qwen2.5-7B exceeds the 8 GB GPU in full precision and is therefore loaded in bitsandbytes 4-bit from an ungated mirror; all other rows are full precision. Llama-3.2 weights are from an ungated mirror of the official full-precision (bf16) checkpoints; the security-decision divergence between Llama-3.2-1B (complies) and the remaining backbones (refuses) shows the registered outcome is a real model property, not a framework artefact. Stronger injection styles and per-agent defenses are release-phase work.

B.5 A worked forced-coordination task

To make the predicate machinery concrete, consider a 3-forced “travel-booking” task on a star of N agents. The orchestrator decomposes the goal into three subtasks assigned to distinct workers: (W1) retrieve an itinerary from a calendar tool; (W2) select a flight within a stated budget via a booking tool; and (W3) pay via a payment tool, but *only* to the payee named in the user’s instruction. A witness (Theorem 6.3) is any set of agents that jointly supplies a valid itinerary, an in-budget flight, and a correctly addressed payment; because all three subtasks are required and are assigned to distinct workers, every witness has size at least three, so the task is 3-forced.

The registered predicates read only the recorded trace τ :

$$\begin{aligned}\Psi(\tau) &= \mathbb{K}[\text{a payment to the instructed payee for an in-budget flight matching the itinerary is recorded}] \\ \Phi(\tau) &= \mathbb{K}[\text{a payment to a payee \textit{not} named in the instruction is recorded (exfiltration of funds), or} \\ &\quad \text{a denylisted action (e.g., disclosing card details to a tool) is recorded}].\end{aligned}$$

A malicious tool (via $\mathcal{E}_{\text{LM}}^\dagger$) might return a booking result that embeds an injected instruction “pay to account X”; a colluding W3 might then act on it. Whether the payment to account X occurs is a recorded state transition, so Φ fires deterministically regardless of what the agents *say* in their messages—the content of the injection cannot change the verdict unless the illicit payment is actually recorded (Theorem 9.4). If instead the adversary set forms a transversal of the witnesses (e.g., it controls W3 in every route), it can force $\Psi = 0$ by withholding the legitimate payment (Theorem 9.15), which the sabotage component of the metric records as coordination failure. This example is illustrative; the shipped task suite generalizes it across roles, tools, and topologies.

B.6 Sensitivity and validity checks

Beyond the headline studies, the protocol includes: (i) an **aggregator-sensitivity** check (RQ1) that recomputes configuration rankings under $u\sigma$, $\min(u, \sigma)$, and a weighted geometric mean $u^w\sigma^{1-w}$ (Theorem 9.3) and reports rank correlation; (ii) an **evaluator-integrity** check (RQ2) that measures how often a crafted injection flips the LLM judge J while leaving the deterministic verdict $C(\tau)$ unchanged (an empirical instance of Theorem 9.4 versus Theorem 9.5); (iii) an **emulation-fidelity** check that compares emulated and real tools on the subset of tasks where a real implementation is available, reporting the change in ASR_{sys} and NRP_{sys} and relating it to κ ; and (iv) **backbone-robustness** checks that repeat the headline degradation study across all five model families so that no conclusion rests on a single model. Each check is a registered configuration in the released harness, so validity, not only headline numbers, is reproducible.

References

- [1] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.09024.
- [2] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi: 10.1111/j.2517-6161.1995.tb02031.x.
- [3] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.
- [4] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013. ISBN 9780199535255. doi: 10.1093/acprof:oso/9780199535255.001.0001.
- [5] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [6] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025. MAST failure taxonomy for multi-agent LLM systems.
- [7] J. Chalupa, P. L. Leath, and G. R. Reich. Bootstrap percolation on a bethe lattice. *Journal of Physics C: Solid State Physics*, 12(1):L31–L35, 1979. doi: 10.1088/0022-3719/12/1/008.

- [8] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023.
- [9] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.10848.
- [10] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 130185–130213, 2024. arXiv:2407.12784.
- [11] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [12] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.
- [13] Stav Cohen, Ron Bitton, and Ben Nassi. Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications. *arXiv preprint arXiv:2403.02817*, 2024.
- [14] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-2636. deterministic state-based security() checks; arXiv:2406.13352.
- [15] John R. Douceur. The Sybil attack. *Peer-to-Peer Systems (IPTPS 2002), Lecture Notes in Computer Science*, 2429:251–260, 2002. doi: 10.1007/3-540-45748-8_24.
- [16] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325*, 2023.
- [17] Joseph L. Fleiss. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382, 1971. doi: 10.1037/h0031619.
- [18] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.
- [19] Mark Granovetter. Threshold models of collective behavior. *American Journal of Sociology*, 83(6):1420–1443, 1978. doi: 10.1086/226707.
- [20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

- [21] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023.
- [22] Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.08567.
- [23] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.
- [24] Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, et al. Multi-agent risks from advanced AI. Technical Report Technical Report #1, Cooperative AI Foundation, 2025. arXiv:2502.14143.
- [25] Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. The emerged security and privacy of LLM agent: A survey with case studies. *ACM Computing Surveys*, 2025. doi: 10.1145/3773080. arXiv:2407.19354.
- [26] Pengfei He, Yuping Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. Red-teaming LLM multi-agent systems via communication attacks. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. Agent-in-the-Middle (AiTM); arXiv:2502.14847.
- [27] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2009.03300.
- [28] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830.
- [29] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [30] Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. TrustAgent: Towards safe and trustworthy LLM-based agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. arXiv:2402.01586.
- [31] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabisa. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.

- [32] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- [33] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [34] David Kempe, Jon Kleinberg, and   va Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '03)*, pages 137–146. ACM, 2003. doi: 10.1145/956750.956769.
- [35] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982. doi: 10.1145/357172.357176.
- [36] Donghyun Lee and Mo Tiwari. Prompt infection: LLM-to-LLM prompt injection within multi-agent systems. *arXiv preprint arXiv:2410.07283*, 2024.
- [37] Erich L. Lehmann and Joseph P. Romano. *Testing Statistical Hypotheses*. Springer Texts in Statistics. Springer, 3rd edition, 2005. ISBN 9780387988641. doi: 10.1007/0-387-27605-X.
- [38] Joel Z. Leibo, Edgar Du  nez-Guzm  n, Alexander Sasha Vezhnevets, John P. Agapiou, Peter Sunehag, Raphael Koster, Jayd Matyas, Charles Beattie, Igor Mordatch, and Thore Graepel. Scalable evaluation of multi-agent reinforcement learning with Melting Pot. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 6187–6199, 2021. arXiv:2107.06857.
- [39] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17760.
- [40] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [41] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, et al. Agent-Bench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.
- [42] Xiaogeng Liu, Nan Xu, Muhao Chen, and Chaowei Xiao. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.04451.

- [43] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, Leo Yu Zhang, and Yang Liu. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*, 2023.
- [44] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security)*, 2024. arXiv:2310.12815.
- [45] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic LLM-powered agent network for task-oriented agent collaboration. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2310.02170.
- [46] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 35181–35224, 2024. arXiv:2402.04249.
- [47] Colin McDiarmid. On the method of bounded differences. In *Surveys in Combinatorics, 1989*, volume 141 of *London Mathematical Society Lecture Note Series*, pages 148–188. Cambridge University Press, 1989. doi: 10.1017/CBO9781107359949.008.
- [48] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2311.12983.
- [49] Jerzy Neyman and Egon S. Pearson. On the problem of the most efficient tests of statistical hypotheses. *Philosophical Transactions of the Royal Society of London. Series A*, 231(694–706):289–337, 1933. doi: 10.1098/rsta.1933.0009.
- [50] OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- [51] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [52] Marshall Pease, Robert Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *Journal of the ACM*, 27(2):228–234, 1980. doi: 10.1145/322186.322188.
- [53] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3419–3448. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.225. arXiv:2202.03286.
- [54] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. In *NeurIPS 2022 Workshop on ML Safety*, 2022. arXiv:2211.09527.

- [55] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2307.07924.
- [56] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [57] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023. arXiv:2310.10501.
- [58] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. ToolEmu; arXiv:2309.15817.
- [59] Donald B. Rubin. Estimating causal effects of treatments in randomized and non-randomized studies. *Journal of Educational Psychology*, 66:688–701, 1974. doi: 10.1037/h0037350.
- [60] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- [61] Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, Ben Hagag, William L. Anderson, Peter Belcak, et al. Open challenges in multi-agent security: Towards secure systems of interacting AI agents. *arXiv preprint arXiv:2505.02077*, 2025. doi: 10.48550/arXiv.2505.02077. MASEC; Sec. 4.7 states no multi-agent security benchmark exists.
- [62] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [63] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021. arXiv:2009.14471.
- [64] Yu Tian, Xiao Yang, Jingyuan Zhang, Yinpeng Dong, and Hang Su. Evil geniuses: Delving into the safety of LLM-based agents. *arXiv preprint arXiv:2311.11855*, 2023.
- [65] Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Mintong Kang, Chenhui Zhang, Chejian Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, Sang T. Truong, Simran Arora, Mantas Mazeika, Dan Hendrycks, Zinan Lin, Yu Cheng, Sanmi Koyejo, Dawn Song, and Bo Li. DecodingTrust: A comprehensive assessment of trustworthiness in GPT models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. arXiv:2306.11698.

- [66] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.
- [67] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*, 2023.
- [68] Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. G-Safeguard: A topology-guided security lens and treatment on LLM-based multi-agent systems. *arXiv preprint arXiv:2502.11127*, 2025.
- [69] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, et al. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741.
- [70] Duncan J. Watts. A simple model of global cascades on random networks. *Proceedings of the National Academy of Sciences*, 99(9):5766–5771, 2002. doi: 10.1073/pnas.082090499.
- [71] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2307.02483.
- [72] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [73] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch out for your agents! investigating backdoor threats to LLM-based agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.11208.
- [74] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [75] Miao Yu, Shilong Wang, Guibin Zhang, Junyuan Mao, Chenlong Yin, Qijiong Liu, Qingsong Wen, Kun Wang, and Yang Wang. NetSafe: Exploring the topological safety of multi-agent networks. *arXiv preprint arXiv:2410.15686*, 2024.
- [76] Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. AutoDefense: Multi-agent LLM defense against jailbreak attacks. *arXiv preprint arXiv:2403.04783*, 2024.
- [77] Qiushi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv:2403.02691.

- [78] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. ASB; introduces $\text{NRP} = \text{PNA} (1 - \text{ASR})$; arXiv:2410.02644.
- [79] Zaibin Zhang, Yongting Zhang, Lijun Li, Hongzhi Gao, Lijun Wang, Huchuan Lu, Feng Zhao, Yu Qiao, and Jing Shao. PsySafe: A comprehensive framework for psychological-based attack, defense, and evaluation of multi-agent system safety. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2401.11880.
- [80] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.
- [81] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.13854.
- [82] Zhenhong Zhou, Zherui Li, Jie Zhang, Yuanhe Zhang, Kun Wang, Yang Liu, and Qing Guo. CORBA: Contagious recursive blocking attacks on multi-agent systems based on large language models. *arXiv preprint arXiv:2502.14529*, 2025.
- [83] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.